FACULDADE DE INFORMÁTICA
DE PRESIDENTE PRUDENTE

MANUAL DO SISTEMA

PROGRAMA DE ESTÁGIO SUPERVISIONADO

LUCAS SAS ORTIZ CAMPOS
[LION FITNESS]
GLAUCO ANTONIO RUIZ
02/2026

Sumário

1. INTRODUÇÃO....3
1.1 Objetivo....3
1.2 Escopo....3
1.3 Definições, Siglas e Abreviações....6
1.4 Referências....6
1.5 Informações Adicionais....7
1.5.1 Dados da Instituição....7
1.5.2 Dados da Empresa....7
1.6 Visão Geral....8
2. REQUISITOS DE SOFTWARE....9
2.1 Lista de Funções/Estórias de Usuários....9
2.2 Modelo Conceitual de Classes....13
2.3 Especificação das Estórias de Usuário....14
3. PROJETO DE SOFTWARE....62
3.1 Diagrama de Classes Final....62
3.2 Diagrama Entidade-Relacionamento Final....63
4. PROCEDIMENTOS DE INSTALAÇÃO....64
5. REFERÊNCIAS....68

1. INTRODUÇÃO

1.1 Objetivo

Este Documento de Especificação de Requisitos de Software (ERS) tem como objetivo descrever, de forma clara, completa e verificável, os requisitos funcionais e não funcionais do sistema a ser desenvolvido para a academia **LION FITNESS**, localizada no município de Santo Anastácio – SP.

O documento formaliza a proposta de construção de um sistema de gestão destinado a apoiar e padronizar as operações administrativas da academia, abrangendo principalmente o controle de alunos, planos, pagamentos, inadimplência, fichas de treino e avaliações físicas. Dessa forma, a ERS estabelece um acordo entre as partes envolvidas (cliente e desenvolvedor), reduzindo ambiguidades e garantindo rastreabilidade dos requisitos ao longo do desenvolvimento.

Além de orientar a implementação, este documento também tem a finalidade de servir como referência para validação do sistema junto ao cliente, permitindo que as funcionalidades entregues estejam alinhadas às necessidades reais da academia e contribuam para a melhoria dos processos internos.

1.2 Escopo

A **LION FITNESS** é uma academia voltada à prática de musculação e condicionamento físico, oferecendo planos de acesso e acompanhamento profissional para alunos da cidade de Santo Anastácio e região. No contexto de operação diária, a academia necessita manter controle organizado de informações essenciais, tais como dados cadastrais dos alunos, plano contratado, vigência de pagamento, situação de inadimplência e registros técnicos relacionados ao acompanhamento do aluno, como fichas de treino e avaliações físicas.

Atualmente, a academia utiliza ferramentas digitais para apoio às suas atividades operacionais. No entanto, o sistema proposto neste documento representa uma solução estruturada e formalmente especificada, com foco em organização administrativa, centralização das informações e definição clara das regras de negócio, podendo atuar de forma complementar ou evolutiva em relação às ferramentas existentes.

O sistema proposto, denominado **Sistema de Gestão da Academia LION FITNESS** (nome provisório), será uma ferramenta central para apoiar a administração e o atendimento da academia, visando aumentar a eficiência operacional, melhorar a confiabilidade das informações e permitir maior controle gerencial do negócio.

- **Gestão de Cadastros (CRUD):** Gerenciamento completo de Alunos (com CPF único e upload de foto em formato **BYTEA**), Usuários do sistema, Personal Trainers e Planos (Mensal e Diária).

- **Gestão Financeira:** Controle de assinaturas e pagamentos manuais e online (com integração técnica via Mercado Pago para Pix e cartões, testada em ambiente local utilizando mocks e simulações sem homologação em produção ou sandbox externo real), com automatização do **cálculo de vigência** baseado no atributo **duration_days** e identificação de inadimplência.

- **Gestão Técnica e Acompanhamento:** Criação de fichas de treino utilizando um catálogo interno de exercícios ou exercícios customizados. Inclui interface dedicada para visualização do aluno e painel exclusivo para o Personal Trainer.

- **Relatórios e Interface:** Emissão de relatórios gerenciais (novos alunos, cancelamentos, ativos/inativos e inadimplentes) e interface moderna com suporte a **tema claro e escuro**.

- **Regras de Negócio Implementadas:** Bloqueio de cancelamento para alunos com dívidas. Regra de vigência calculada rigorosamente com base na quantidade de dias configurada no plano (**duration_days**).

Fora do escopo (nesta versão inicial)

Não fazem parte do escopo da versão **ATUAL** do sistema (mantidos no backlog para evoluções futuras):

- avaliação física e registro de carga/peso por aparelho;
- agendamento de aulas e reserva de aparelhos;
- funcionalidades mobile nativas e integração biométrica;
- cálculo de multa ou juros por atraso.

Com a implementação deste sistema, espera-se melhorar a organização e a rastreabilidade das informações, aumentar a qualidade do atendimento aos alunos e fornecer apoio ao controle financeiro e administrativo da academia.

1.3 Definições, Siglas e Abreviações

- **ERS:** Especificação de Requisitos de Software
- **RF_B:** Requisito Funcional Básico (operações CRUD)
- **RF_F:** Requisito Funcional Fundamental (transações de negócio)
- **RF_S:** Requisito Funcional de Saída (consultas, relatórios, dashboards)
- **CRUD:** Create, Read, Update, Delete (Criar, Consultar, Atualizar, Excluir/Inativar)
- **UML:** Unified Modeling Language (Linguagem de Modelagem Unificada)
- **DER:** Diagrama Entidade-Relacionamento
- **SGBD:** Sistema Gerenciador de Banco de Dados
- **BD:** Banco de Dados
- **HTTPS:** Hypertext Transfer Protocol Secure
- **API:** Application Programming Interface
- **CPF:** Cadastro de Pessoas Físicas (identificador único do aluno no sistema)

1.4 Referências

Nº	Título	Data Aquisição	Responsável pelo Fornecimento
1	Entrevista com responsável da LION FITNESS	2026	Administração da Academia
2	Modelo de ERS – Faculdade de Informática de Presidente Prudente	2026	
3	Política interna de pagamentos e cancelamentos – LION FITNESS	2026	Administração da Academia

1.5 Informações Adicionais

1.5.1 Dados da Instituição

A Faculdade de Informática de Presidente Prudente (FIPP) é uma instituição de ensino superior dedicada à formação de profissionais na área de Tecnologia da Informação, oferecendo cursos de graduação e especialização voltados ao desenvolvimento de competências técnicas, gerenciais e científicas.

O presente projeto está vinculado à disciplina de **Estágio Supervisionado**, componente curricular obrigatório do curso de Sistemas de Informação, cujo objetivo é proporcionar ao aluno a aplicação prática dos conhecimentos adquiridos ao longo da formação acadêmica.

O desenvolvimento do sistema proposto integra as atividades do estágio, permitindo a vivência real de um projeto de engenharia de software, incluindo levantamento de requisitos, modelagem, documentação, implementação e validação junto ao cliente.

Curso: Sistemas de Informação
Disciplina: Estágio Supervisionado
Aluno: Lucas Sas Ortiz Campos
Professor Orientador: Glauco Antonio Ruiz

Ano: 2026

1.5.2 Dados da Empresa

A **LION FITNESS** é uma academia localizada no município de Santo Anastácio – SP, voltada à prática de musculação e condicionamento físico. A instituição oferece planos de acesso nas modalidades **mensal** e **diária**, atendendo alunos da cidade e região, contando com o acompanhamento técnico de profissionais capacitados.

A academia realiza atividades relacionadas à elaboração de fichas de treino, avaliações físicas periódicas e orientação individualizada aos alunos. Além das atividades técnicas, também executa rotinas administrativas essenciais para sua operação, como controle de cadastros, gerenciamento de planos, registro de pagamentos e acompanhamento da situação financeira dos alunos.

Atualmente, a academia utiliza ferramentas digitais para apoio às suas atividades operacionais. Contudo, parte dos registros técnicos ainda é realizada de forma manual, especialmente no que se refere a fichas de treino em papel, o que pode dificultar o acompanhamento histórico das informações, a padronização dos registros e a centralização dos dados.

Diante desse cenário, o sistema de gestão proposto neste documento tem como objetivo modernizar e estruturar o controle administrativo e técnico da academia, promovendo maior organização, rastreabilidade e eficiência. A digitalização das fichas de treino permitirá manter histórico consolidado das informações, reduzir perdas de dados e facilitar o acesso tanto por profissionais quanto pelos próprios alunos (em modo consulta). Destaca-se que a funcionalidade de avaliação física automatizada faz parte do escopo futuro/backlog do sistema.

O sistema visa atuar como uma solução organizada e modelada segundo boas práticas de engenharia de software, com foco em:

- Centralização e padronização das informações cadastrais;
- Automatização do cálculo de vigência de planos;
- Identificação estruturada de inadimplência;
- Controle formal de cancelamentos;
- Substituição gradual de registros manuais por registros digitais;
- Geração de relatórios gerenciais para apoio à tomada de decisão.

Estrutura Organizacional (Simplificada)

A estrutura organizacional da LION FITNESS pode ser representada da seguinte forma:

- Administração Geral
    - Profissionais/Instrutores
    - Personais (Personal Trainers)

No contexto do sistema:

- **Profissionais da academia** serão responsáveis pela gestão administrativa e técnica das informações.
- **Alunos** terão acesso restrito ao sistema para consulta de seus próprios dados (fichas de treino e situação do plano), além de realizarem operações de autosserviço financeiro e renovação de planos disponibilizadas em seu painel. Avaliações físicas e registros persistentes de cargas permanecem fora do escopo atual.
- **Personais (Personal Trainers):** poderão acessar o sistema para consultar e acompanhar os treinos e registros técnicos dos alunos sob sua responsabilidade, sem permissão para alterar dados administrativos ou financeiros.

1.6 Visão Geral

Este documento está estruturado de forma a apresentar, de maneira clara e organizada, os requisitos do sistema proposto para a LION FITNESS.

O **Capítulo 1** contém a introdução do documento, incluindo objetivo, escopo, definições e informações institucionais relevantes ao projeto.

O **Capítulo 2** apresenta os requisitos de software, organizados na forma de Estórias de Usuário, devidamente classificadas em Requisitos Funcionais Básicos, Fundamentais e de Saída, contemplando regras de negócio, critérios de aceite e modelagens necessárias.

O **Capítulo 3** descreve os artefatos de modelagem do sistema, incluindo Diagrama de Classes e Diagrama Entidade-Relacionamento, consolidando a estrutura técnica da solução proposta.

O **Capítulo 4** apresenta os procedimentos necessários para instalação e utilização do sistema.

Por fim, são listadas as referências utilizadas como base para a elaboração desta Especificação de Requisitos de Software.

2. REQUISITOS DE SOFTWARE

2.1 Lista de Funções/Estórias de Usuários

As estórias de usuário estão classificadas e priorizadas de acordo com a complexidade e impacto no negócio, alinhando-se com a abordagem ágil de desenvolvimento. O sistema implementado opera com os perfis de acesso: **ADMIN**, **PERSONAL_TRAINER**, **OPERATIONAL**.

- **Funções Básicas (CRUDs):** referem-se às operações de Cadastro, Consulta, Atualização e Inativação necessárias para a gestão de entidades do sistema.
- **Funções Fundamentais:** referem-se às transações de negócio e funcionalidades centrais, incluindo autenticação e controle de permissões.
- **Funções de Saída:** referem-se às funções que geram informações de saída relevantes (consultas, relatórios e dashboards gerenciais).

Além da classificação, é indicado a complexidade de implementação da estória, sua situação real (**Implementada** ou **Futura/Backlog**) e o perfil de acesso responsável por sua execução.

1. FUNÇÕES BÁSICAS (CRUDs)

Estória de Usuário	Situação	Parecer do Orientador
Título: Gerenciar alunos (com CPF único, soft delete e upload de foto) **Classificação:** Básica **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Gerenciar profissionais/usuários do sistema (com controle de roles e autenticação) **Classificação:** Básica **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Gerenciar personal trainers **Classificação:** Básica **Complexidade de Implementação:** BAIXA	Implementada	OK
Título: Gerenciar planos (Mensal e Diária) **Classificação:** Básica **Complexidade de Implementação:** BAIXA	Implementada	OK
Título: Gerenciar catálogo interno de exercícios **Classificação:** Básica **Complexidade de Implementação:** BAIXA	Parcialmente implementada	OK
Título: Gerenciar exercícios customizados **Classificação:** Básica **Complexidade de Implementação:** BAIXA	Implementada	OK

Estória de Usuário	Situação	Parecer do Orientador

Título: Autenticação (Login JWT) **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Implementada	OK
Título: Recuperação de senha por e-mail **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Registrar adesão do aluno ao plano **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Registrar pagamento manual **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Calcular vigência automática do plano mensal **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Implementada	OK
Título: Identificar alunos inadimplentes **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Impedir cancelamento com mensalidade vencida **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Registrar cancelamento de aluno **Classificação:** Fundamental **Complexidade de Implementação:** BAIXA	Implementada	OK
Título: Cadastrar ficha de treino (Prescrição de exercícios) **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Implementada	OK
Título: Atualizar ficha de treino mantendo histórico **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Parcialmente implementada	OK
Título: Permitir consulta de dados pelo aluno (Painel do Aluno) **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Controlar permissões por perfil de acesso **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Implementada	OK

FUNÇÕES FUNDAMENTAIS

Estória de Usuário	Situação	Parecer do Orientador
Título: Integrar pagamento via gateway online **Classificação:** Fundamental **Complexidade de**	Implementada tecnicamente	OK

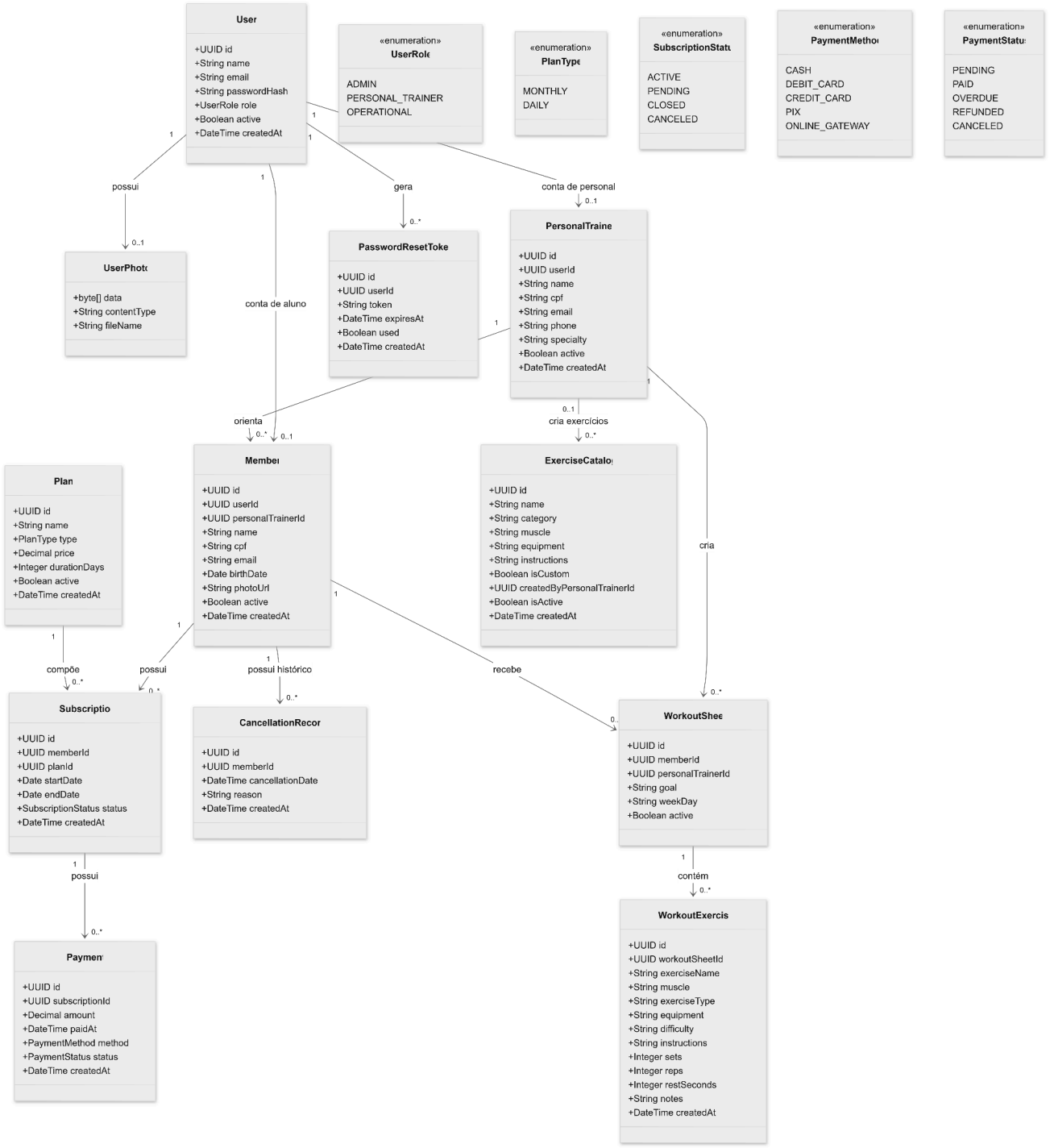
Implementação: ALTA	em ambiente local com mocks e simulações	
Título: Processar confirmação automática de pagamento online **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Implementada tecnicamente em ambiente local com mocks e simulações	OK
Título: Registrar avaliação física **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Pendente	OK
Título: Registrar carga/peso por aparelho **Classificação:** Fundamental **Complexidade de Implementação:** MÉDIA	Pendente	OK
Título: Gerar jobs automáticos (ex: notificação de vigência) **Classificação:** Fundamental **Complexidade de Implementação:** ALTA	Pendente	OK

FUNÇÕES DE SAÍDA

Estória de Usuário	Situação	Parecer do Orientador
Título: Gerar relatório de novos alunos por período **Classificação:** Saída **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Gerar relatório de cancelamentos **Classificação:** Saída **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Gerar relatório de alunos ativos e inativos **Classificação:** Saída **Complexidade de Implementação:** BAIXA	Implementada	OK
Título: Gerar relatório de alunos inadimplentes **Classificação:** Saída **Complexidade de Implementação:** MÉDIA	Implementada	OK
Título: Apresentar dashboards administrativos e indicadores financeiros **Classificação:** Saída **Complexidade de Implementação:** ALTA	Implementada	OK

2.2 Modelo Conceitual de Classes

A seguir é apresentado o modelo conceitual que fornece uma visão clara e compreensível do domínio do sistema, ajudando a entender o que precisa ser implementado.



2.3 Especificação das Estórias de Usuário

Estória 1: Gerenciar alunos

Como administrador do sistema, quero gerenciar os alunos da academia (listar, buscar, cadastrar, alterar e desativar), para manter o controle de alunos cadastrados e suas informações pessoais.

Especificação do Requisito

Deve ser possível listar todos os alunos cadastrados no sistema.
O sistema deve permitir buscar alunos pelo nome ou CPF.
O sistema deve permitir cadastrar novos alunos com os seguintes campos obrigatórios:

- Nome;
- CPF;
- Data de nascimento;
- E-mail;
- Senha inicial de acesso.

O sistema deve permitir opcionalmente cadastrar:

- Foto do aluno;
- Personal trainer responsável.

O CPF deve ser único no sistema.
O sistema não deve permitir dois alunos com o mesmo CPF.
Caso seja realizada tentativa de cadastro com CPF já existente, o sistema deve impedir a operação e informar erro.
O sistema deve permitir atualizar os dados do aluno.
O sistema deve permitir desativar um aluno sem removê-lo fisicamente do banco de dados.
A exclusão deve ocorrer utilizando soft delete através do campo is_active.
O sistema deve criar automaticamente um usuário do tipo OPERATIONAL vinculado ao aluno.
O sistema deve registrar automaticamente a data de criação do aluno.
O sistema deve permitir upload de foto do aluno.
A foto deve possuir validação de formato e tamanho.
As imagens devem ser armazenadas utilizando armazenamento binário (BYTEA) no banco de dados.

Diagrama de Classe do Contexto

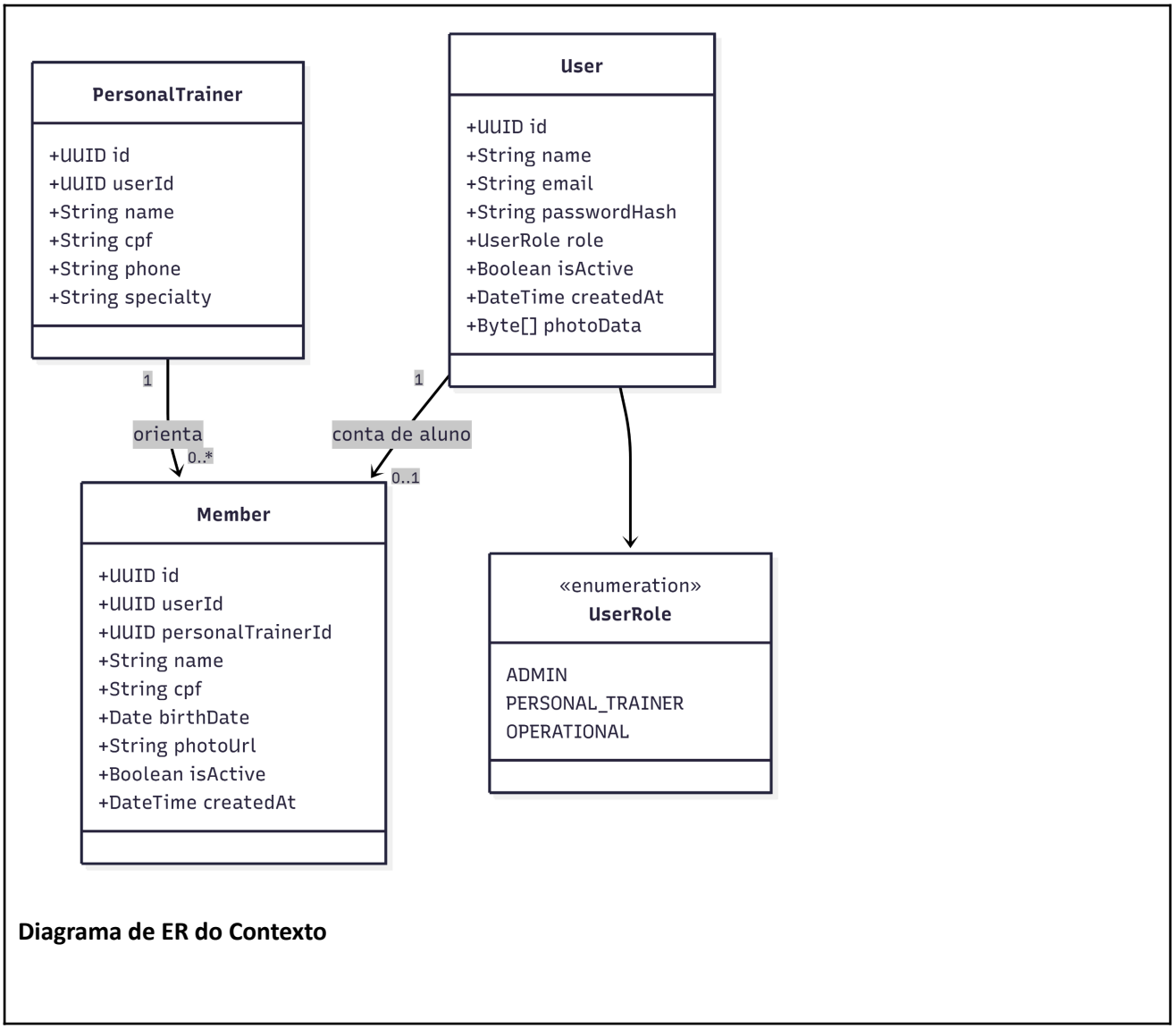
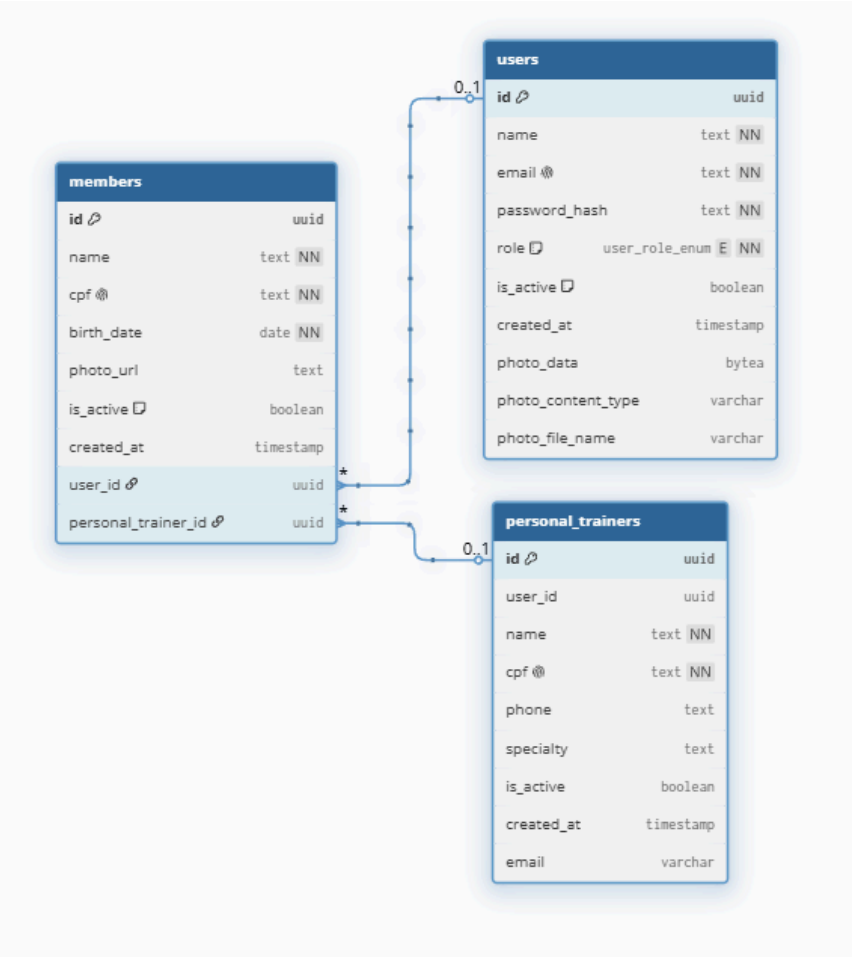


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 2: Gerenciar profissionais/usuários do sistema

Como administrador do sistema, quero gerenciar os usuários e profissionais cadastrados, para controlar o acesso ao sistema conforme o perfil de cada usuário.

Especificação do Requisito

O sistema deve permitir cadastrar usuários administrativos e profissionais.
O sistema deve permitir listar usuários cadastrados.
O sistema deve permitir buscar usuários por nome ou e-mail.
O sistema deve permitir atualizar os dados do usuário.
O sistema deve permitir desativar usuários sem removê-los fisicamente do banco de dados.
O sistema deve exigir os seguintes campos obrigatórios:

- Nome;
- E-mail;
- Senha;
- Perfil de acesso.

O e-mail deve ser único no sistema.
As senhas devem ser armazenadas utilizando criptografia BCrypt.
O sistema deve possuir os seguintes perfis:

- ADMIN;
- PERSONAL_TRAINER;
- OPERATIONAL.

Usuários inativos não devem conseguir autenticar no sistema.
O sistema deve registrar automaticamente a data de criação do usuário.

Diagrama de Classe do Contexto

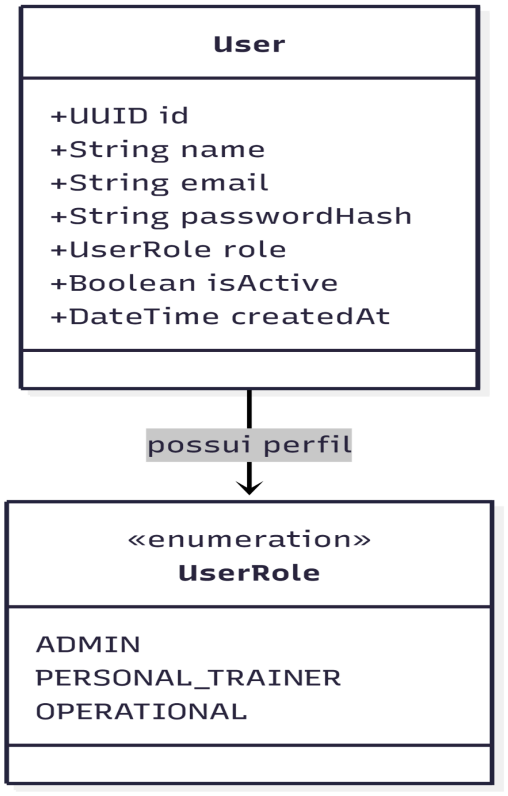
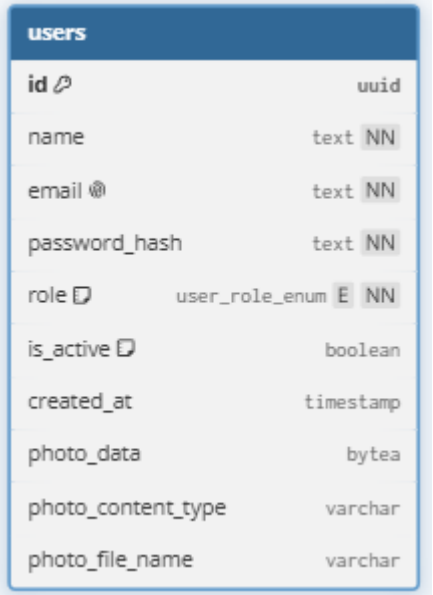


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.1

Estória 3: Gerenciar personal (Personal Trainers)
Como administrador do sistema, quero gerenciar os personal trainers da academia, para controlar os profissionais responsáveis pelo acompanhamento e prescrição de treinos.

Especificação do Requisito

O sistema deve permitir cadastrar personal trainers.

O sistema deve permitir listar os personal trainers cadastrados.

O sistema deve permitir atualizar os dados do personal trainer.

O sistema deve permitir desativar personal trainers sem removê-los fisicamente do banco de dados.

O personal trainer deve possuir vínculo com uma conta de usuário do sistema.

O sistema deve permitir cadastrar:

- Nome;
- CPF;
- E-mail;
- Telefone;
- Especialidade.

O CPF não deve se repetir no sistema.

O personal trainer poderá ser associado aos alunos da academia.

O sistema deve registrar automaticamente a data de criação do personal trainer.

Diagrama de Classe do Contexto

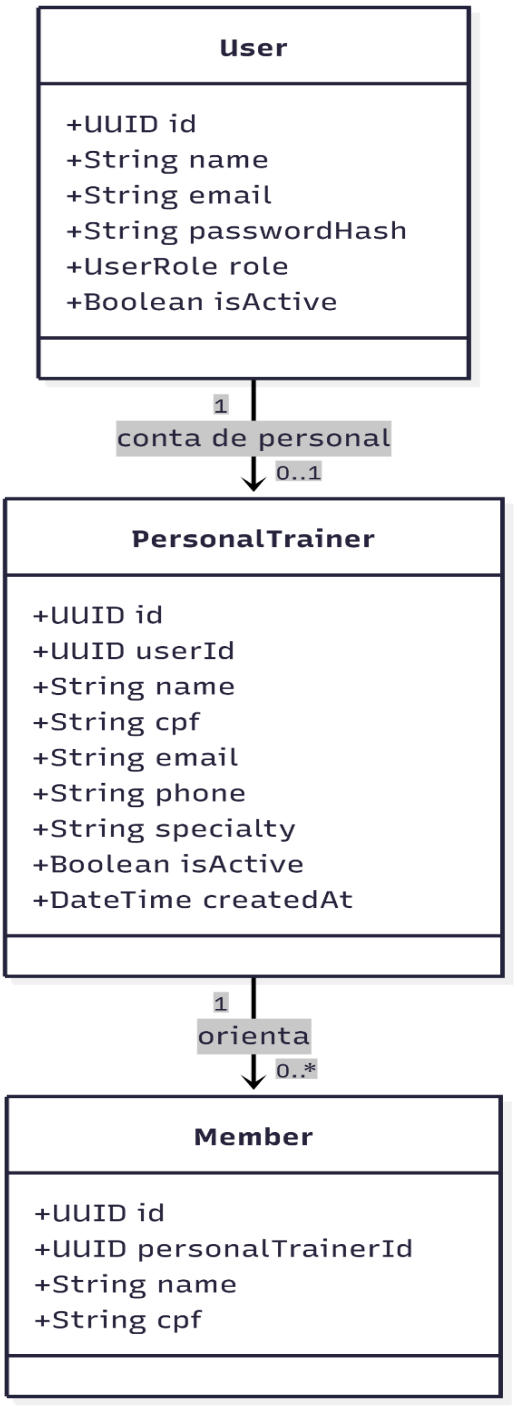
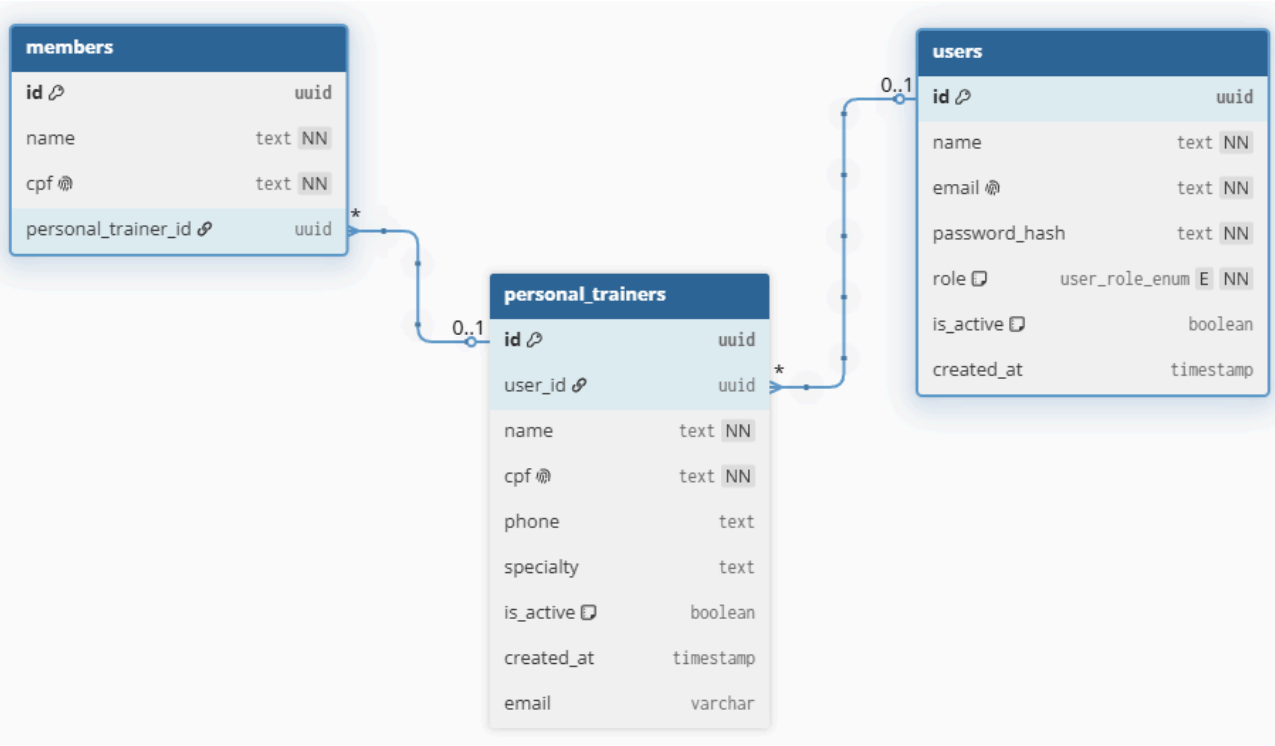


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.1

Estória 4: Gerenciar planos (Mensal e Diária)
Como administrador do sistema, quero gerenciar os planos da academia, para controlar os serviços disponibilizados aos alunos.

Especificação do Requisito

O sistema deve permitir cadastrar planos.

O sistema deve permitir listar os planos cadastrados.

O sistema deve permitir atualizar os dados dos planos.

O sistema deve permitir desativar planos sem removê-los fisicamente do banco de dados.

Cada plano deve possuir:

- Nome;
- Tipo;
- Valor;
- Quantidade de dias;
- Status ativo/inativo.

Os tipos de plano atuais devem ser:

- Mensal;
- Diária.

A duração do plano deve ser utilizada no cálculo automático da vigência da assinatura.

O sistema não deve possuir planos semestrais ou anuais, conforme decisão administrativa da academia.

O sistema deve registrar automaticamente a data de criação do plano.

Diagrama de Classe do Contexto

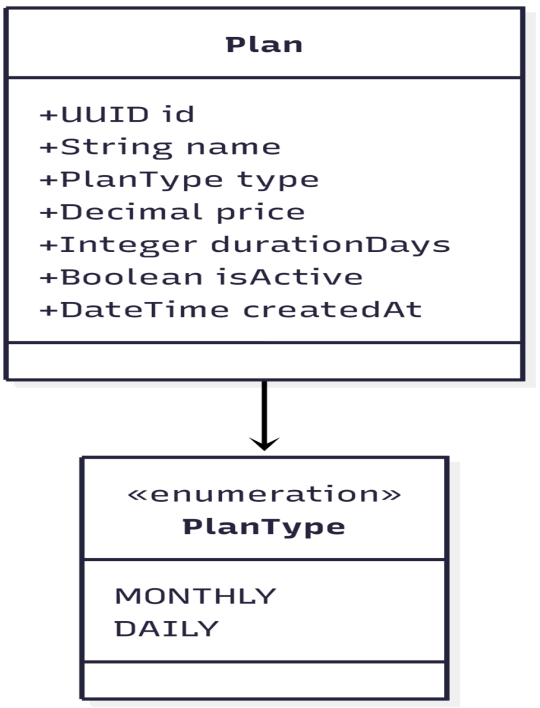
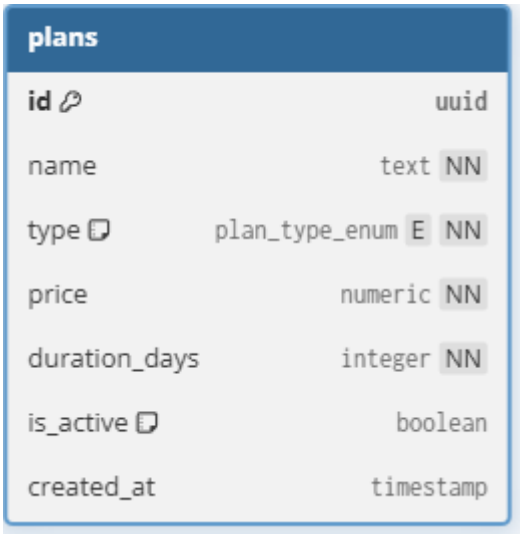


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 5: Gerenciar catálogo interno de exercícios
Como administrador ou personal trainer, quero gerenciar o catálogo interno de exercícios, para utilizar os exercícios cadastrados na criação das fichas de treino.

Especificação do Requisito

O sistema possui um catálogo interno de exercícios parcialmente implementado, cobrindo o cadastro e a listagem estruturada dos exercícios essenciais para a prescrição das fichas de treino.

O sistema deve permitir cadastrar exercícios no catálogo.

O sistema deve permitir listar exercícios cadastrados.

O sistema deve permitir buscar exercícios por:

- Nome;
- Grupo muscular;
- Categoria;
- Equipamento.

Cada exercício deve possuir:

- Nome;
- Categoria;
- Grupo muscular;
- Equipamento;
- Instruções.

O catálogo deve servir como base para a prescrição de treinos.

O sistema deve permitir ativar ou desativar exercícios do catálogo.

A API externa não deve ser tratada como fonte principal de exercícios.

Diagrama de Classe do Contexto

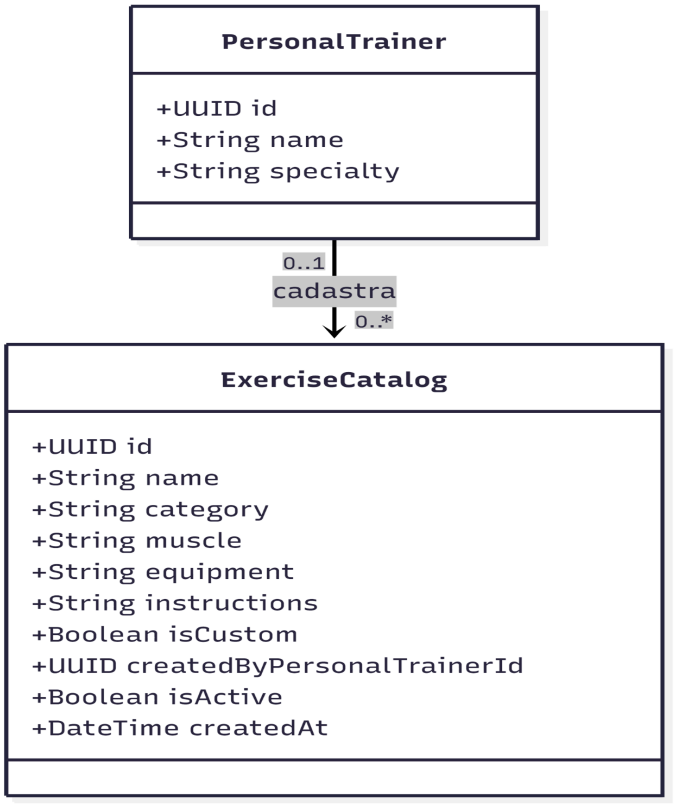
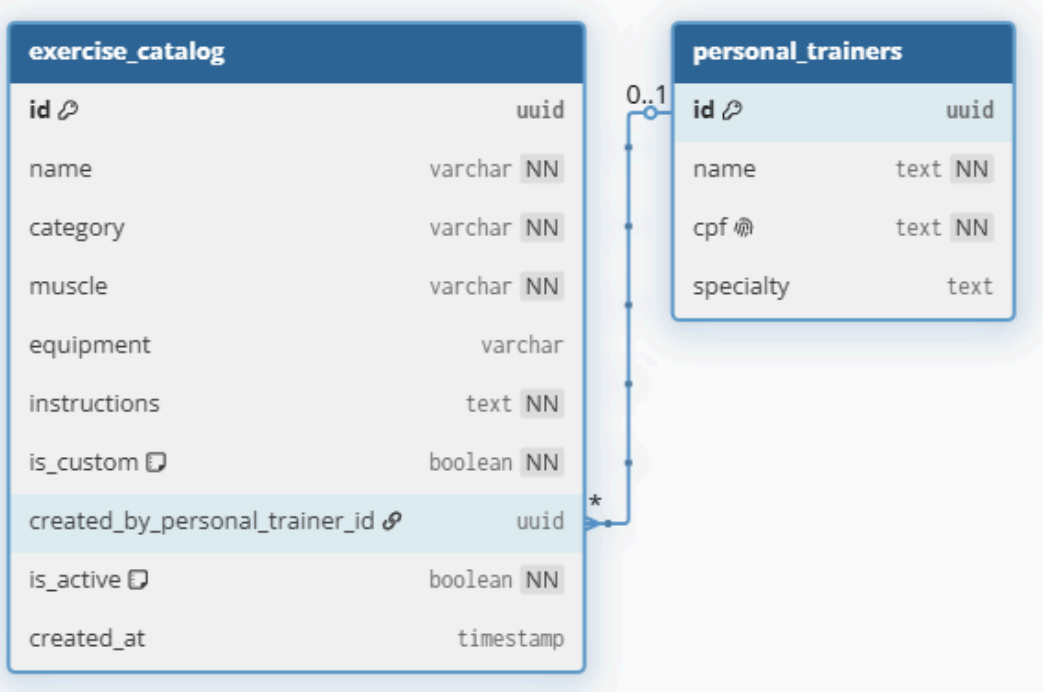


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 6: Gerenciar exercícios customizados
Como personal trainer, quero cadastrar exercícios personalizados, para utilizar exercícios específicos nas fichas de treino dos alunos.

Especificação do Requisito

O sistema deve permitir criar exercícios personalizados.

Exercícios customizados devem ser identificados no catálogo interno.

O exercício personalizado deve possuir vínculo com o personal trainer responsável pela criação.

O sistema deve permitir utilizar exercícios personalizados em fichas de treino.

O sistema deve permitir editar exercícios personalizados.

O sistema deve permitir desativar exercícios personalizados.

O sistema deve registrar automaticamente a data de criação do exercício.

Diagrama de Classe do Contexto

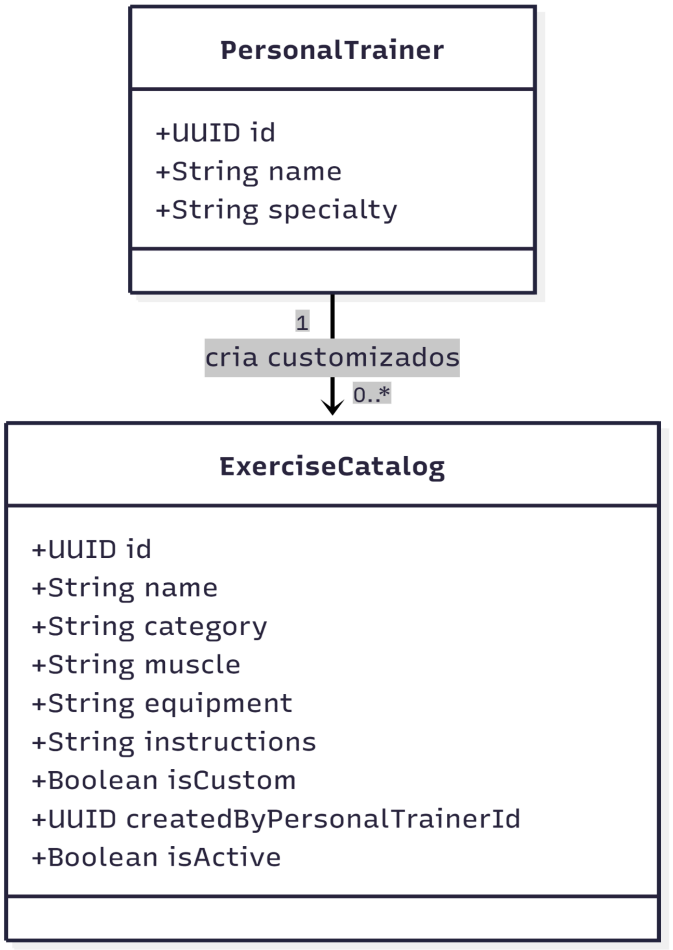
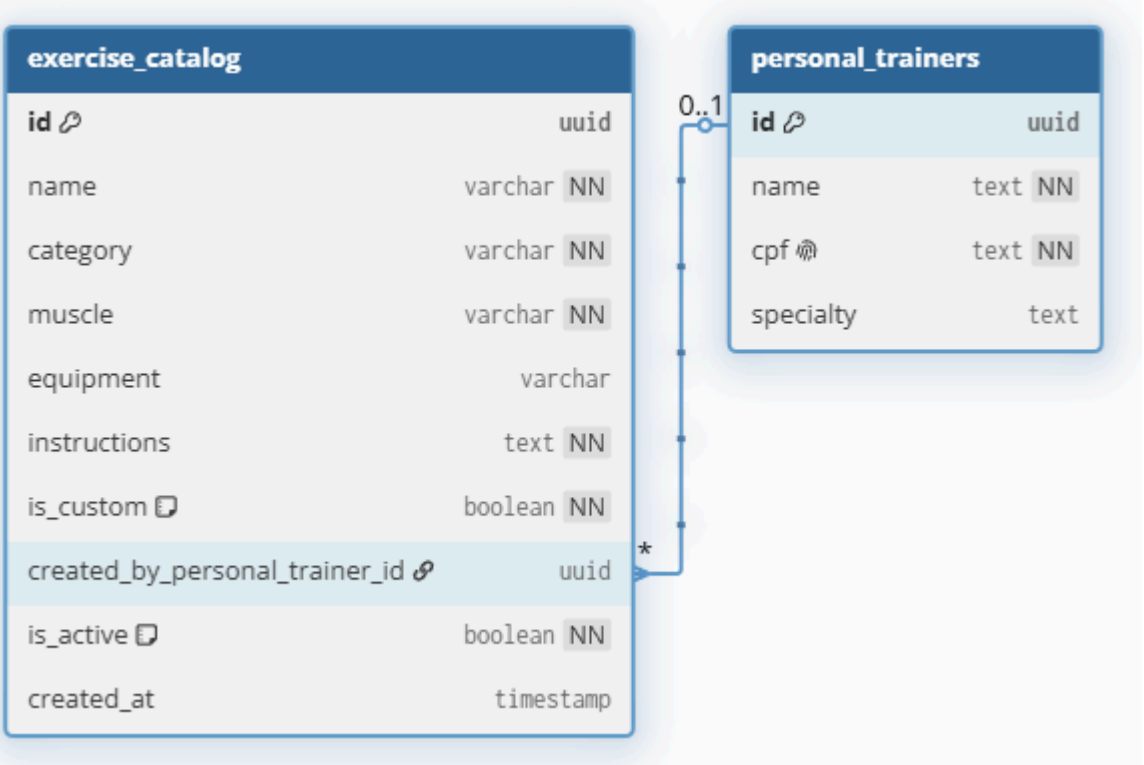


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 7: Autenticação (Login JWT)

Como usuário do sistema, quero realizar autenticação utilizando e-mail e senha, para acessar as funcionalidades conforme meu perfil de acesso.

Especificação do Requisito

O sistema deve permitir autenticação por e-mail e senha.

O sistema deve validar as credenciais informadas.

O sistema deve utilizar criptografia BCrypt para validação das senhas.

O sistema deve gerar token JWT após autenticação válida.

O token JWT deve ser utilizado para acesso às rotas protegidas.

O sistema deve impedir autenticação de usuários inativos.

O sistema deve identificar o perfil do usuário autenticado.

O sistema deve restringir funcionalidades conforme o perfil:

- ADMIN;
- PERSONAL_TRAINER;

- OPERATIONAL.

O sistema deve retornar erro em caso de credenciais inválidas.

Diagrama de Classe do Contexto

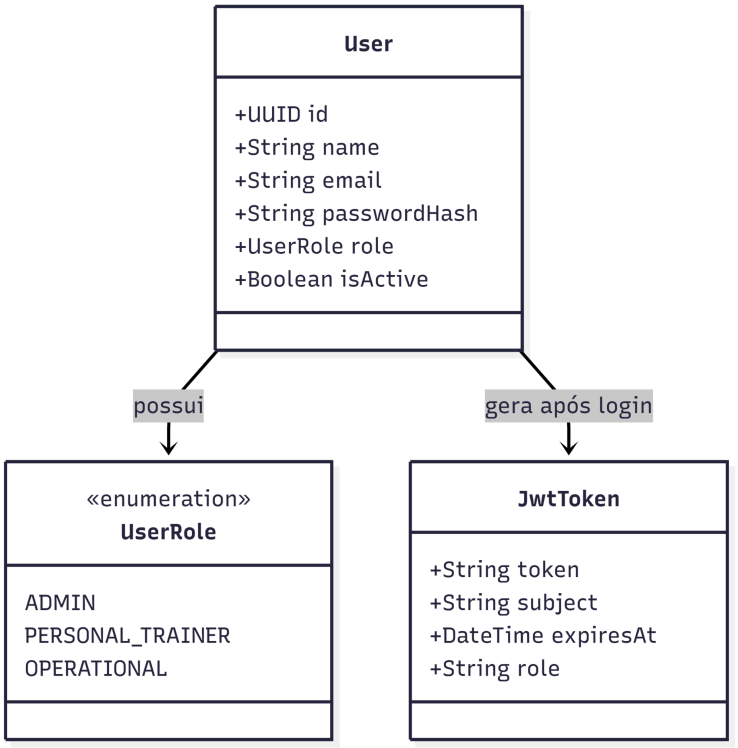


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 8:Recuperação de senha por e-mail

Como usuário do sistema, quero recuperar minha senha através do e-mail cadastrado, para voltar a acessar minha conta em caso de esquecimento.

Especificação do Requisito

O sistema deve permitir solicitar recuperação de senha através do e-mail.

O sistema deve gerar token temporário para redefinição da senha.

O sistema deve enviar link de recuperação por e-mail.

O token deve possuir prazo de validade.

Tokens expirados não devem ser aceitos.

Tokens já utilizados não devem ser reutilizados.

O sistema deve permitir redefinir a senha.

A nova senha deve ser armazenada utilizando BCrypt.

O sistema deve invalidar o token após uso.

Diagrama de Classe do Contexto

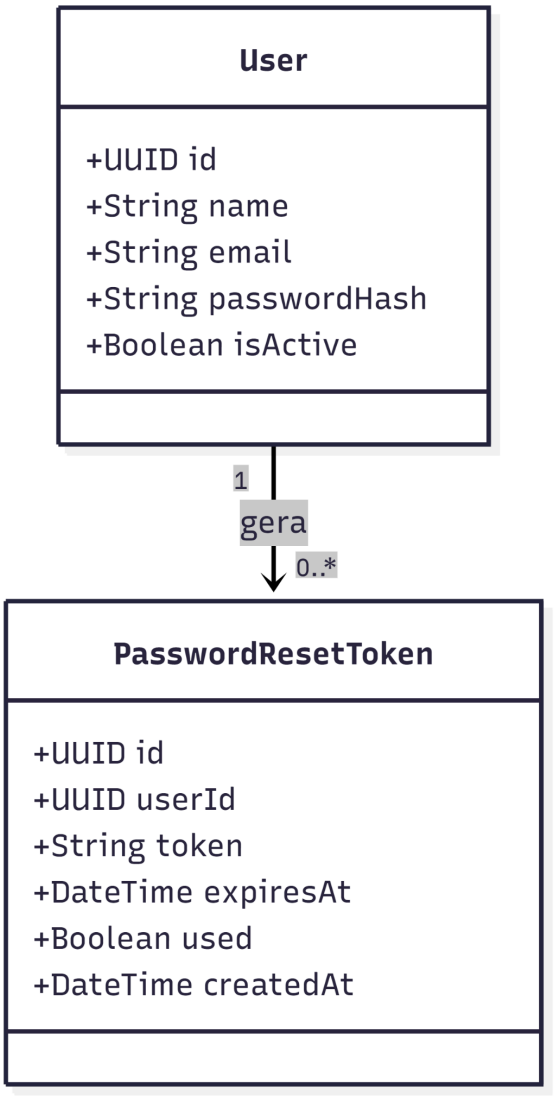
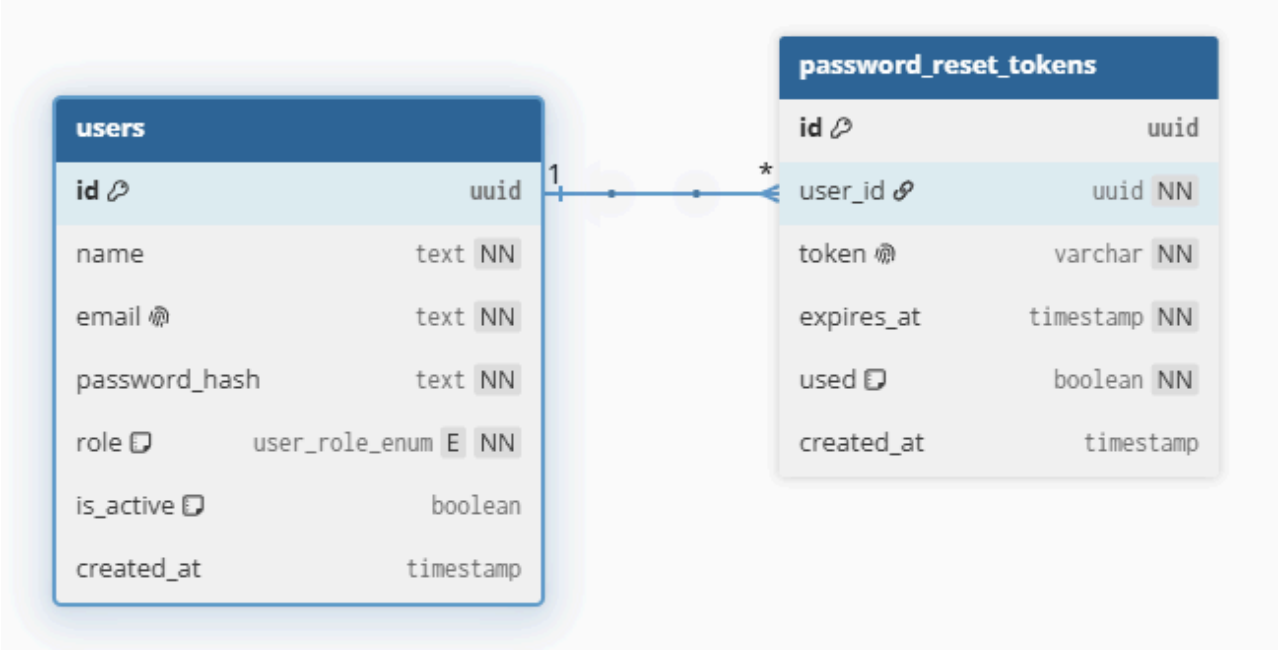


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 9:Registrar adesão do aluno ao plano
Como administrador do sistema, quero registrar a adesão do aluno a um plano, para controlar a vigência e status da assinatura.

Especificação do Requisito

O sistema deve permitir vincular um aluno a um plano.

O sistema deve criar uma assinatura para o aluno.

O sistema deve registrar:

- Data inicial;
- Plano escolhido;
- Status da assinatura.

O sistema deve calcular automaticamente a data final da assinatura.

O cálculo da vigência deve utilizar a duração configurada no plano.

O sistema deve permitir visualizar assinaturas cadastradas.

O sistema deve impedir criação de assinatura para alunos inexistentes.

Diagrama de Classe do Contexto

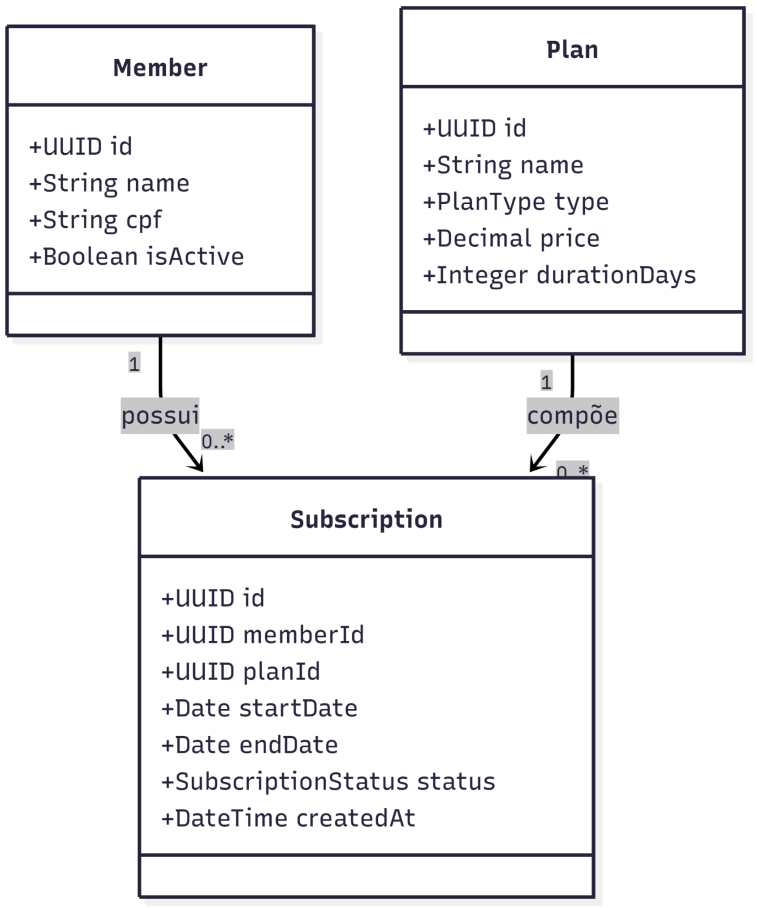
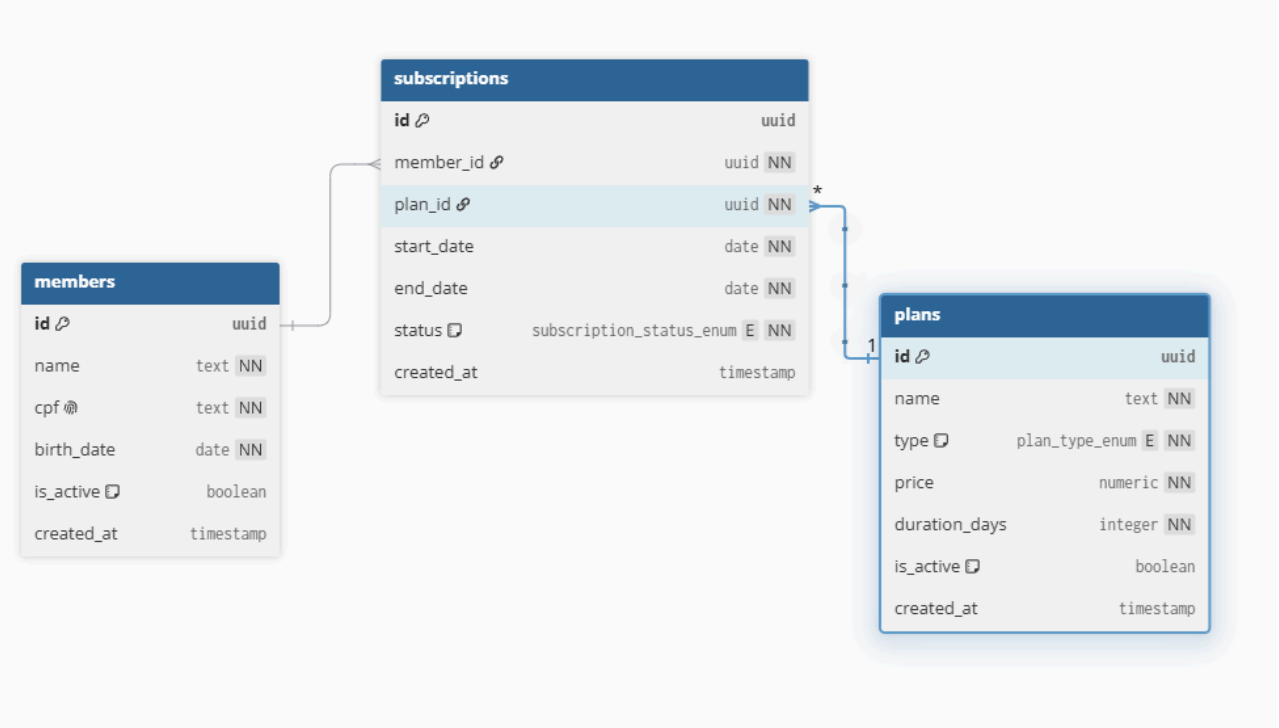


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 10: Registrar pagamento manual
Como administrador do sistema, quero registrar pagamentos manualmente, para controlar os pagamentos

realizados pelos alunos.

Especificação do Requisito

O sistema deve permitir registrar pagamentos manualmente.

O pagamento deve estar vinculado a uma assinatura.

O sistema deve permitir os seguintes métodos de pagamento:

- Dinheiro;
- Pix;
- Cartão;
- Boleto.

O sistema deve registrar:

- Valor;
- Método de pagamento;
- Status;
- Data do pagamento.

O sistema deve possuir os seguintes status:

- Pendente;
- Pago;
- Atrasado;
- Cancelado.

O sistema não deve tratar integração com gateway online como funcionalidade implementada.

O sistema deve permitir consultar pagamentos realizados.

Diagrama de Classe do Contexto

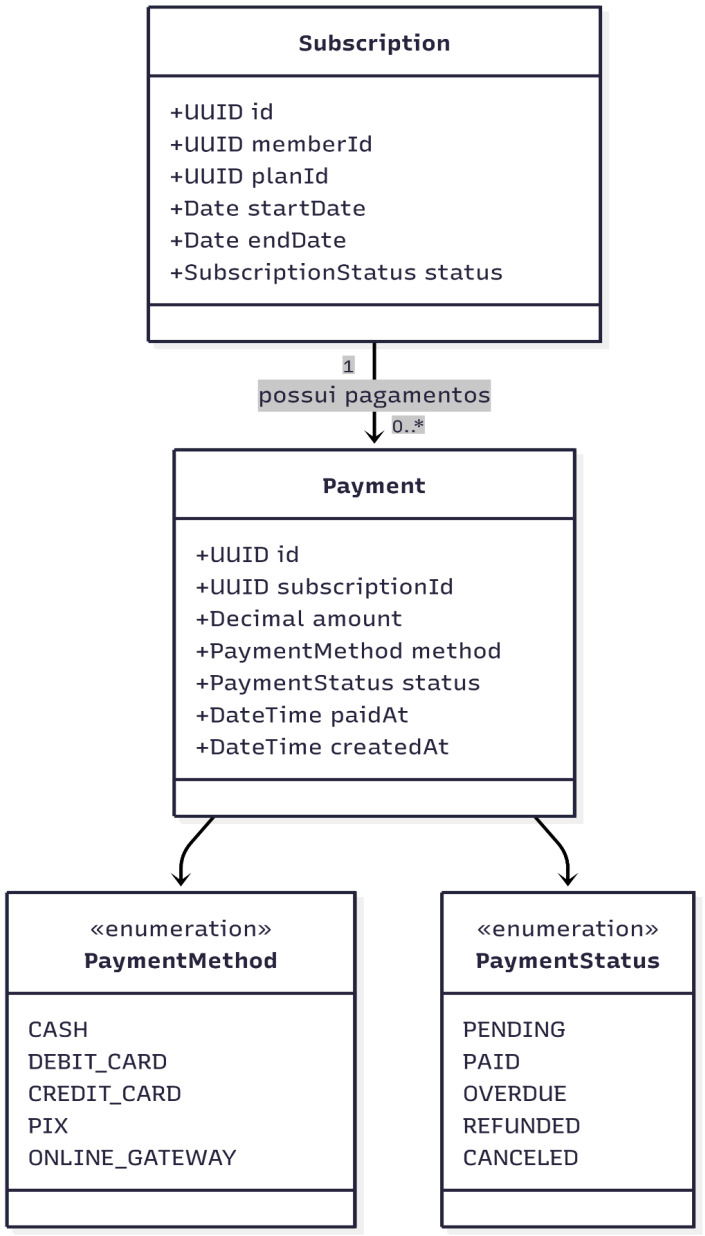
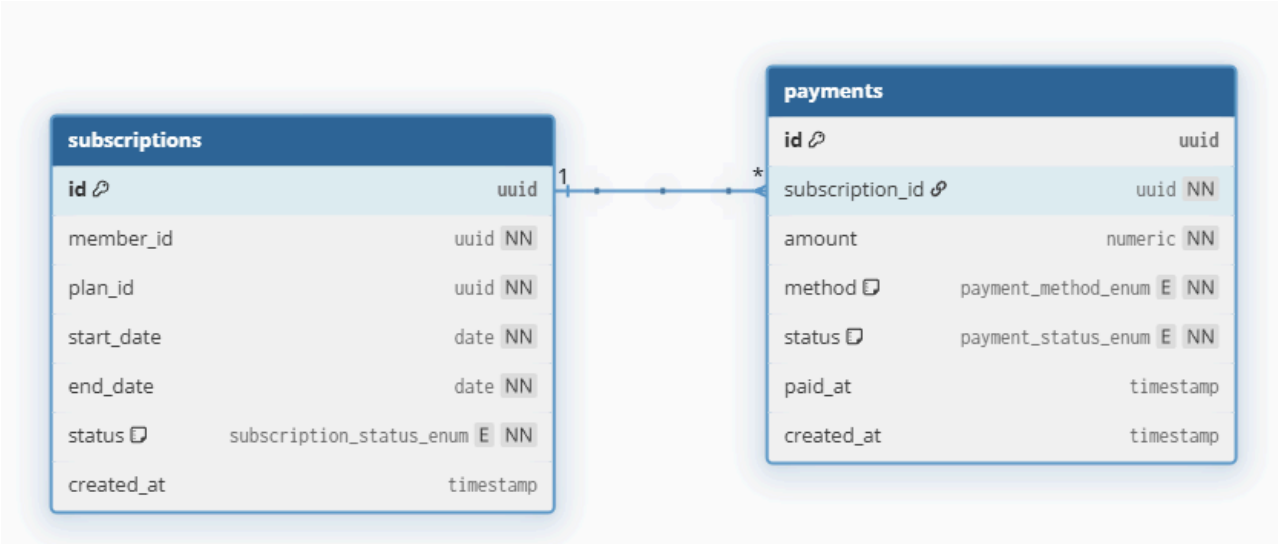


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 11: Calcular vigência automática do plano mensal
Como administrador do sistema, quero que a vigência do plano seja calculada automaticamente, para evitar erros manuais no preenchimento da data final da assinatura.

Especificação do Requisito

O sistema deve calcular automaticamente a data final da assinatura.

O cálculo deve considerar a data inicial informada na adesão.

O cálculo deve considerar a quantidade de dias configurada no plano.

O administrador não deve precisar informar manualmente a data final da assinatura.

Para planos mensais, a vigência deve seguir a duração cadastrada no plano.

Para planos do tipo diária, a vigência deve respeitar a quantidade de dias definida.

O sistema deve armazenar a data inicial e a data final na assinatura.

O sistema deve impedir a criação de assinatura sem plano válido.

O sistema deve impedir a criação de assinatura sem aluno válido.

O status inicial da assinatura deve ser registrado conforme a regra do sistema.

Diagrama de Classe do Contexto

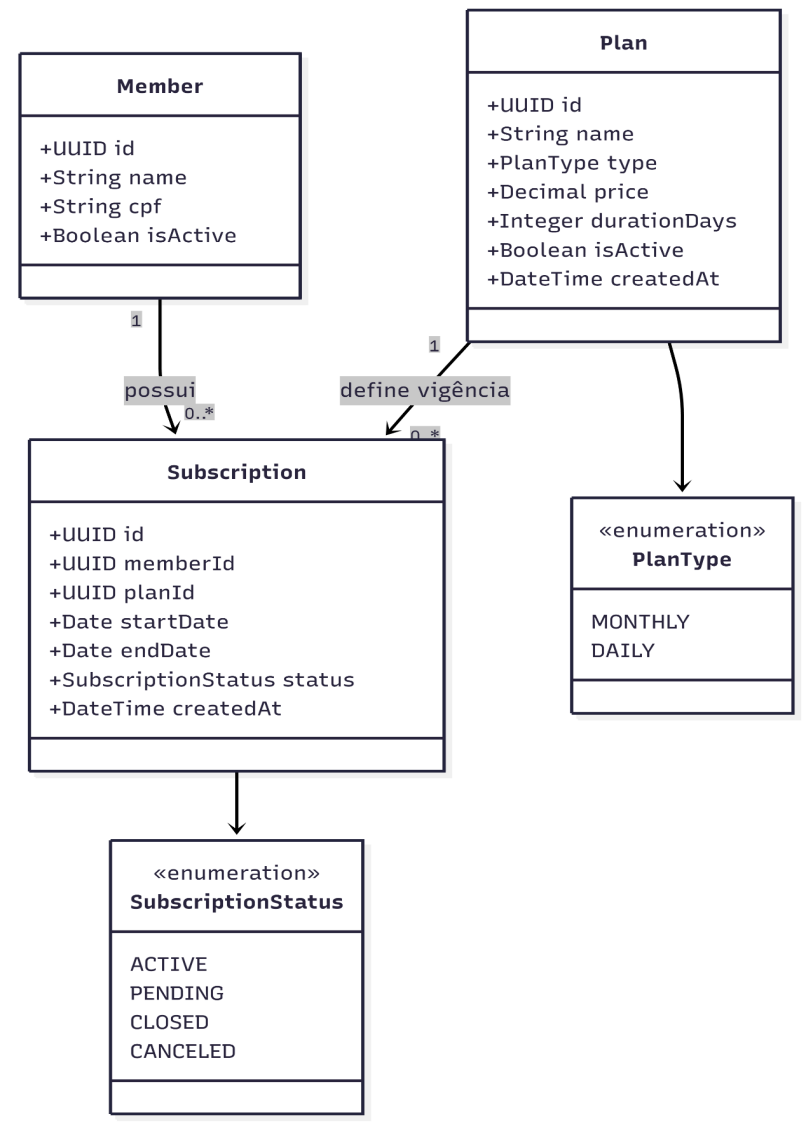


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 12: Identificar alunos inadimplentes
Como administrador do sistema, quero identificar alunos inadimplentes, para acompanhar pendências financeiras e realizar ações de cobrança.

Especificação do Requisito

O sistema deve permitir identificar alunos com pendências financeiras.

Um aluno é considerado inadimplente quando possui pagamentos com status OVERDUE ou PENDING cuja data de término da assinatura associada já expirou.

O sistema deve considerar a situação da assinatura do aluno.

O sistema deve verificar pagamentos vinculados às assinaturas.

O sistema deve permitir listar os alunos inadimplentes.

A listagem deve apresentar informações do aluno, assinatura e pagamento.

O sistema deve apoiar o administrador no controle financeiro.

O sistema não deve aplicar multa ou juros automaticamente.

O sistema deve permitir que a inadimplência seja consultada em relatórios administrativos.

Apenas usuários com perfil ADMIN devem acessar essa funcionalidade.

Diagrama de Classe do Contexto

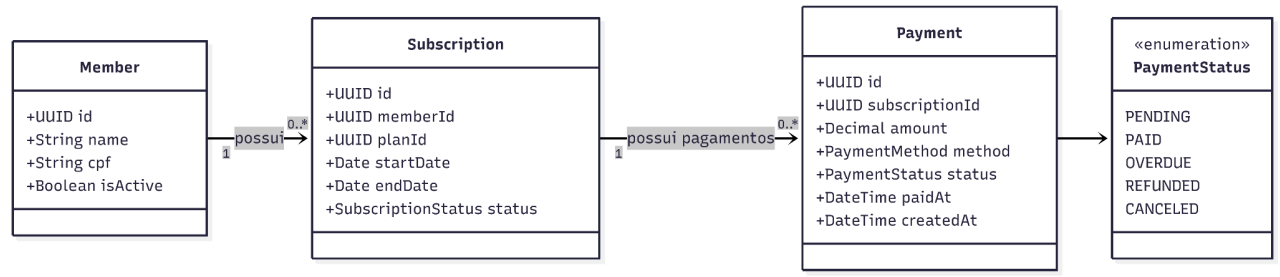
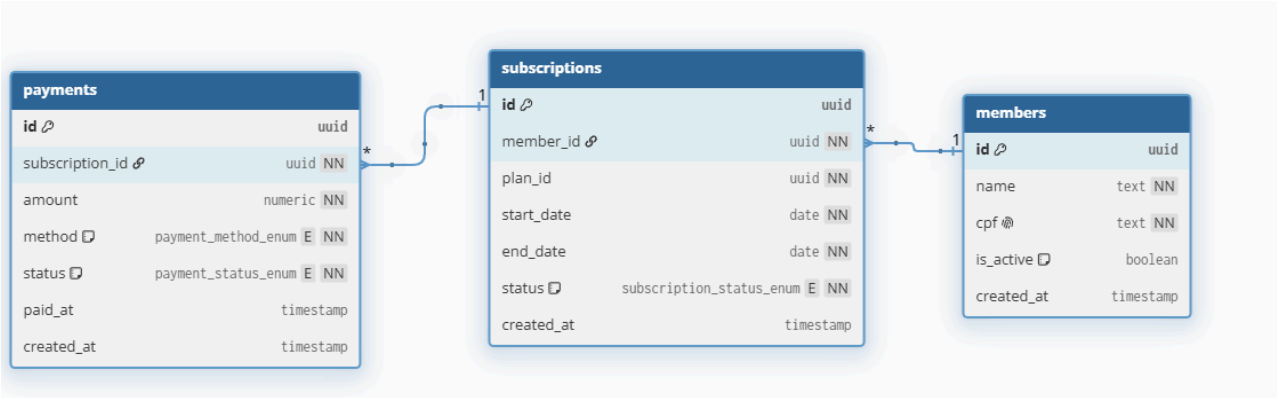


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 13: Impedir cancelamento com mensalidade vencida
Como administrador do sistema, quero que o sistema impeça o cancelamento de alunos com mensalidade vencida, para evitar encerramentos com pendências financeiras.

Especificação do Requisito

O sistema deve validar a situação financeira do aluno antes de permitir a efetivação do cancelamento.

O sistema deve verificar se o aluno possui assinatura vinculada.

O sistema deve verificar se existem cobranças com status PENDING ou OVERDUE vinculadas ao aluno.

Caso exista qualquer pendência financeira (PENDING ou OVERDUE), o cancelamento da assinatura deve ser sumariamente bloqueado pelo sistema.

O sistema deve exibir mensagem informando que há pendência financeira.

O cancelamento só deve ser permitido quando não houver dívida registrada.

A validação deve ocorrer antes da alteração do status do aluno.

A validação deve ocorrer antes da criação do registro de cancelamento.

Apenas usuários ADMIN devem poder executar essa ação.

Diagrama de Classe do Contexto

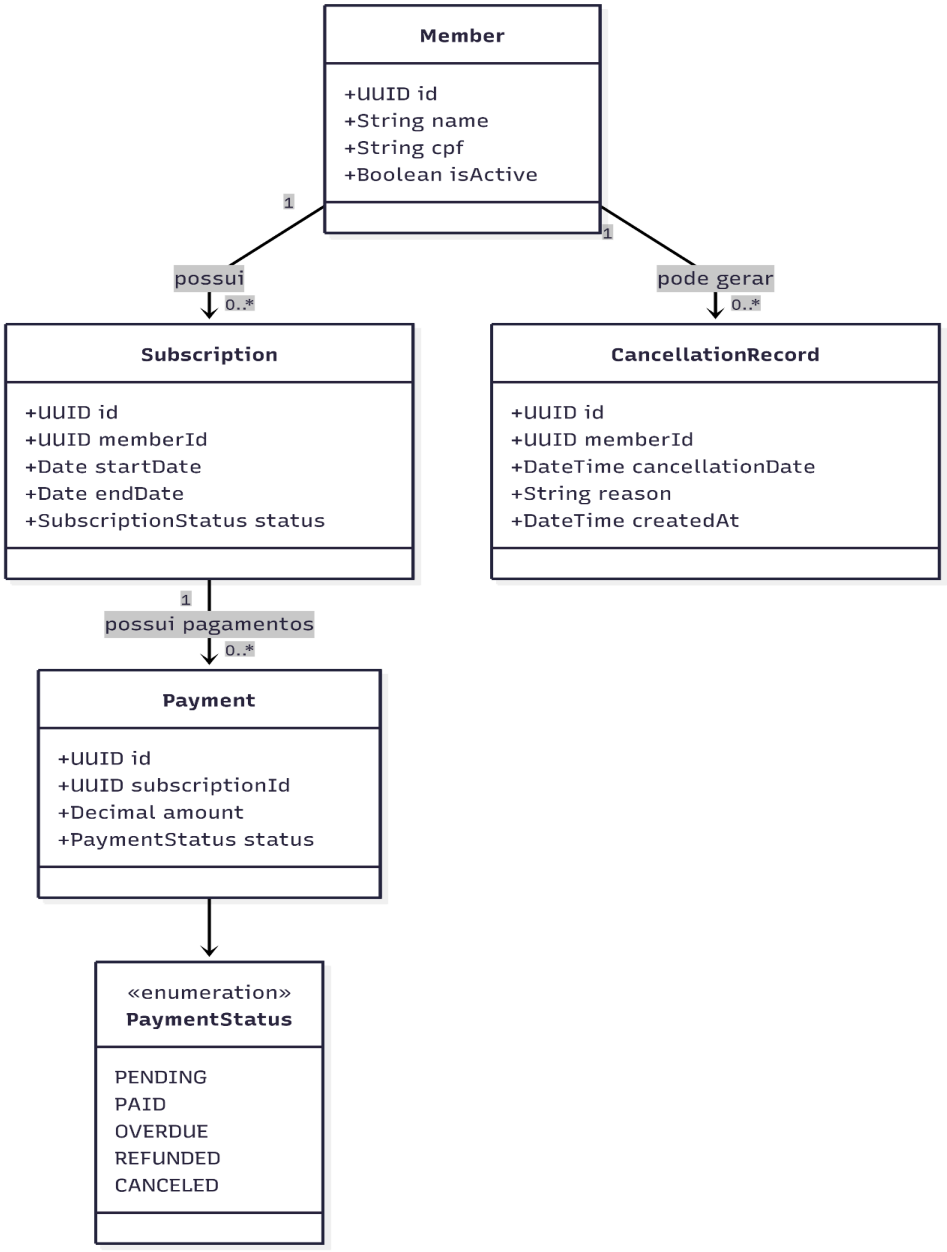
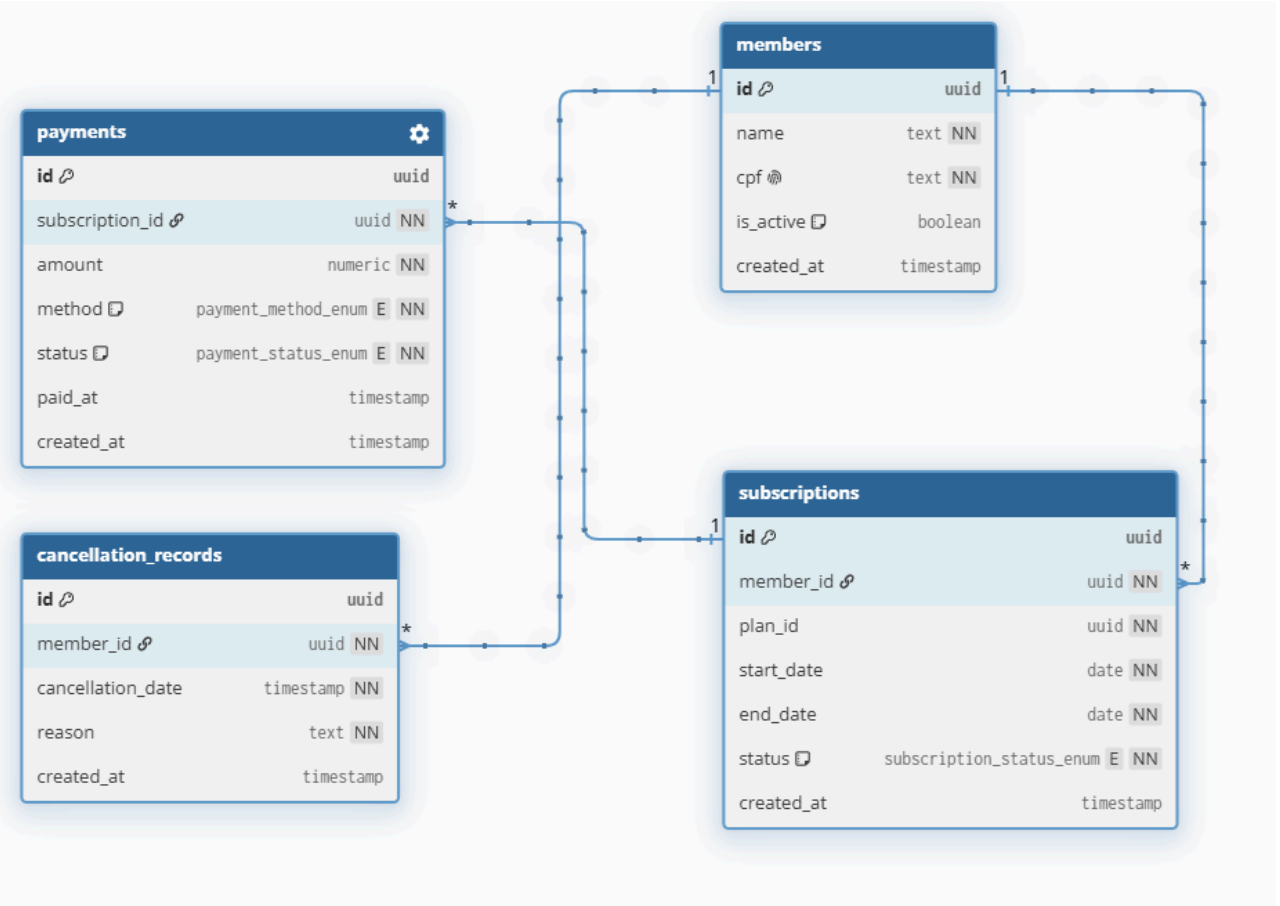


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 14: Registrar cancelamento de aluno
Como administrador do sistema, quero registrar o cancelamento de um aluno, para manter histórico de desligamentos e motivos de evasão.

Especificação do Requisito

O sistema deve permitir cancelar/desativar um aluno.

O cancelamento deve ser realizado apenas por usuário ADMIN.

O sistema deve exigir o motivo do cancelamento.

O sistema deve registrar a data do cancelamento.

O aluno cancelado deve ser desativado logicamente.

O sistema não deve remover fisicamente o aluno do banco de dados.

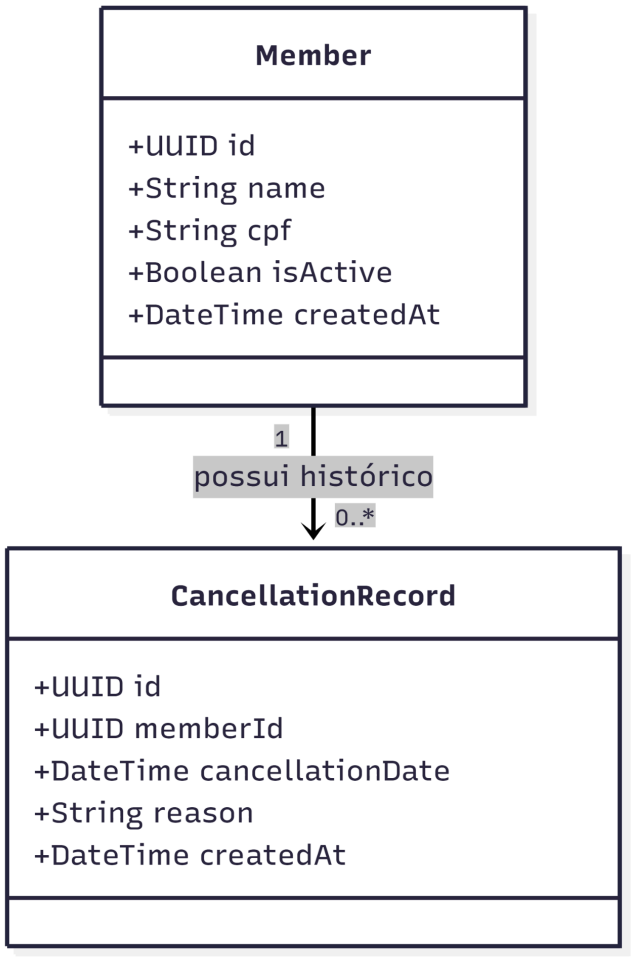
O registro do cancelamento deve ser armazenado em histórico.

O histórico deve conter aluno, data e motivo do cancelamento.

O cancelamento deve preservar os dados anteriores do aluno.

Os dados de cancelamento devem ser utilizados posteriormente em relatórios.

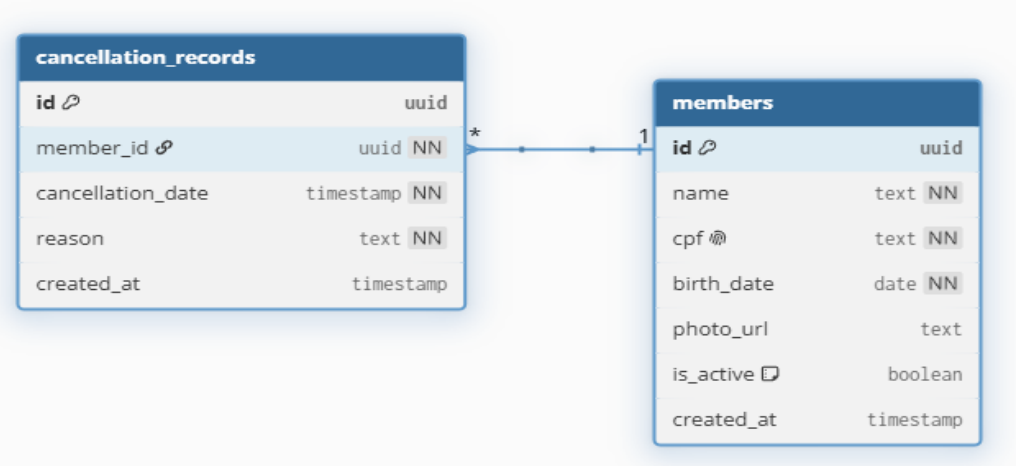
Diagrama de Classe do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Diagrama de ER do Contexto



Estória 15: Cadastrar ficha de treino
Como personal trainer, quero cadastrar fichas de treino para meus alunos, para organizar a prescrição de exercícios conforme o objetivo e o dia da semana.

Especificação do Requisito

O sistema deve permitir cadastrar fichas de treino.

A ficha de treino deve estar vinculada a um aluno.

A ficha de treino deve estar vinculada a um personal trainer.

O personal trainer deve poder criar fichas apenas para seus alunos vinculados.

A ficha deve possuir objetivo do treino.

A ficha pode ser organizada por dia da semana.

O sistema deve permitir adicionar exercícios à ficha.

Os exercícios devem possuir:

- Nome;
- Grupo muscular;
- Tipo de exercício;
- Equipamento;
- Dificuldade;
- Instruções;
- Séries;
- Repetições;
- Tempo de descanso;
- Observações.

O sistema deve utilizar o catálogo interno de exercícios como base para a prescrição.

O aluno deve poder consultar sua ficha posteriormente no painel do aluno.

Diagrama de Classe do Contexto

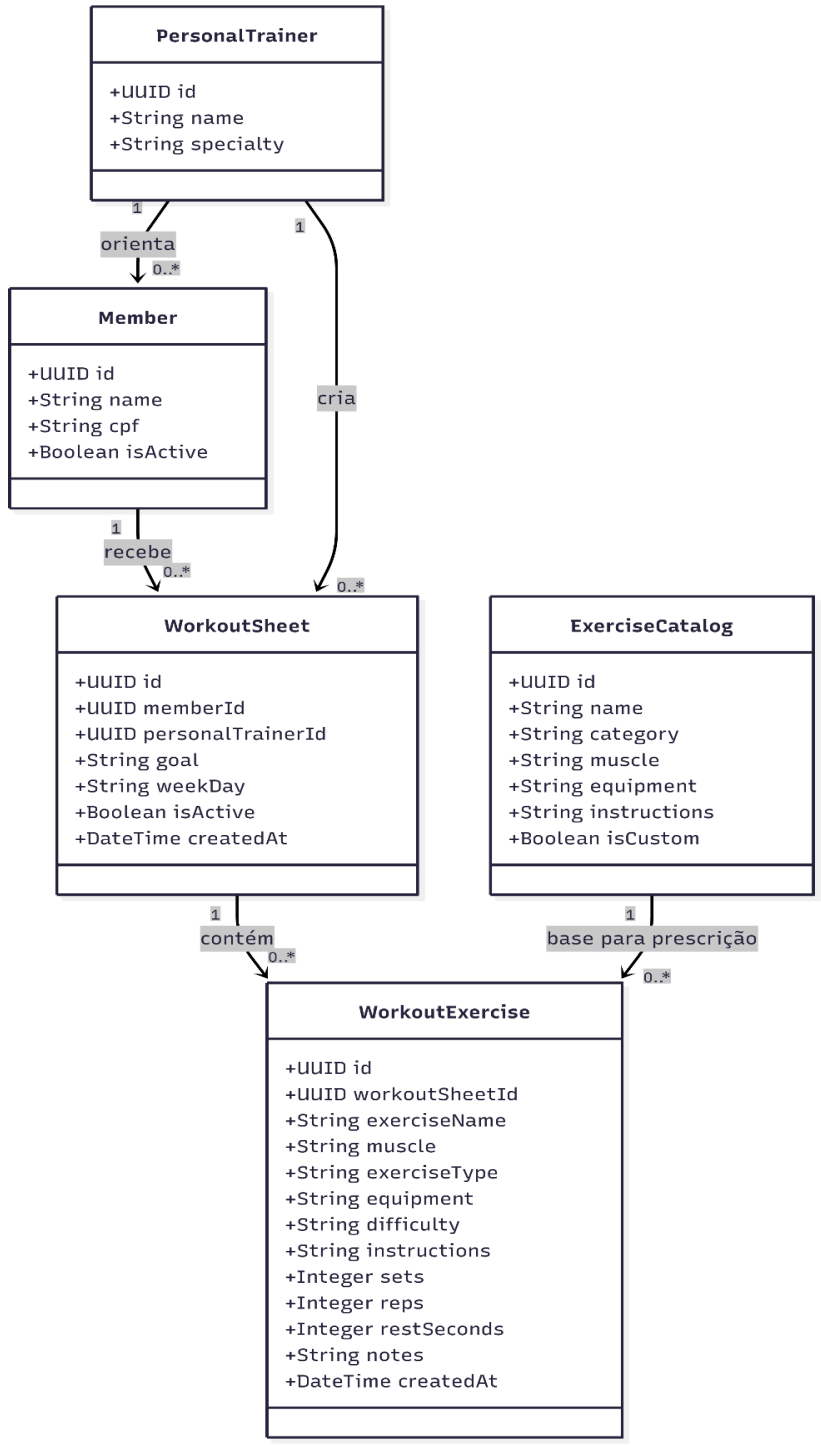
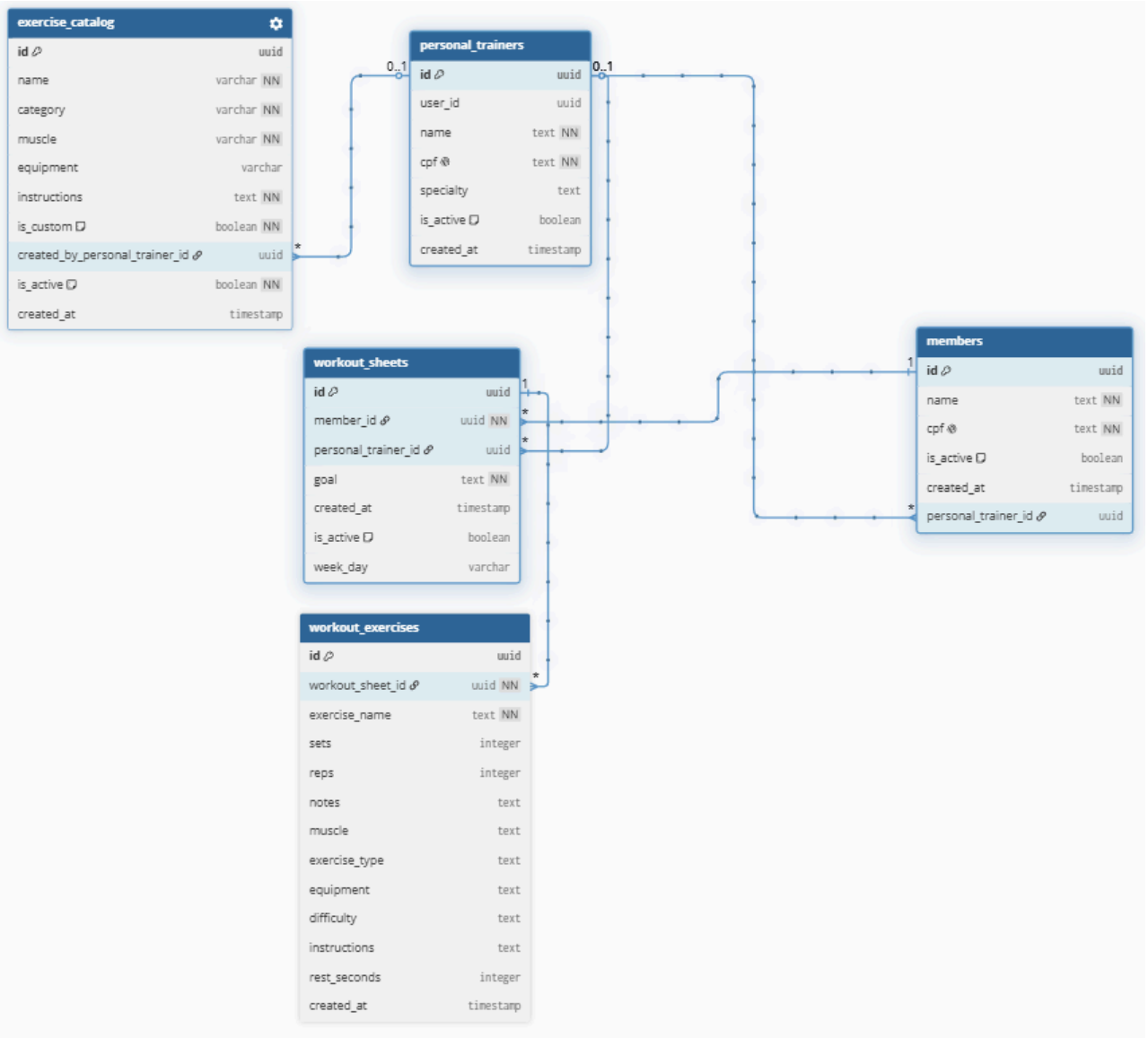


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 16: Atualizar ficha de treino mantendo histórico
Como personal trainer, quero atualizar fichas de treino mantendo histórico das alterações, para acompanhar a evolução dos alunos sem perder registros anteriores.

Especificação do Requisito

O sistema deve permitir atualizar fichas de treino existentes.

A ficha deve estar vinculada a um aluno.

A ficha deve estar vinculada a um personal trainer.

O sistema deve permitir alterar:

- Objetivo do treino;
- Dia da semana;
- Exercícios;
- Séries;

- Repetições;
- Tempo de descanso;
- Observações.

A funcionalidade de atualização de fichas está parcialmente implementada: as alterações nas fichas de treino ativas são realizadas diretamente, enquanto o versionamento completo em tabela de histórico (workout_sheet_history) representa uma evolução da estrutura do backend.

O histórico deve registrar a ficha alterada.

O histórico deve registrar a data da alteração.

O histórico deve permitir informar o motivo da alteração.

Fichas antigas podem permanecer registradas para consulta histórica.

Apenas ADMIN e PERSONAL_TRAINER devem atualizar fichas de treino no sistema.

Diagrama de Classe do Contexto

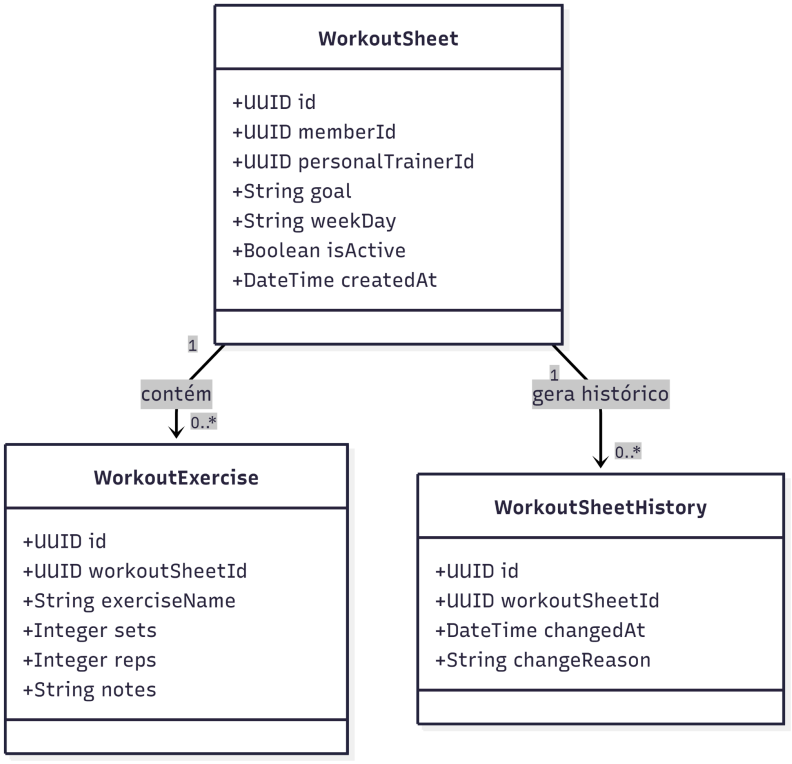
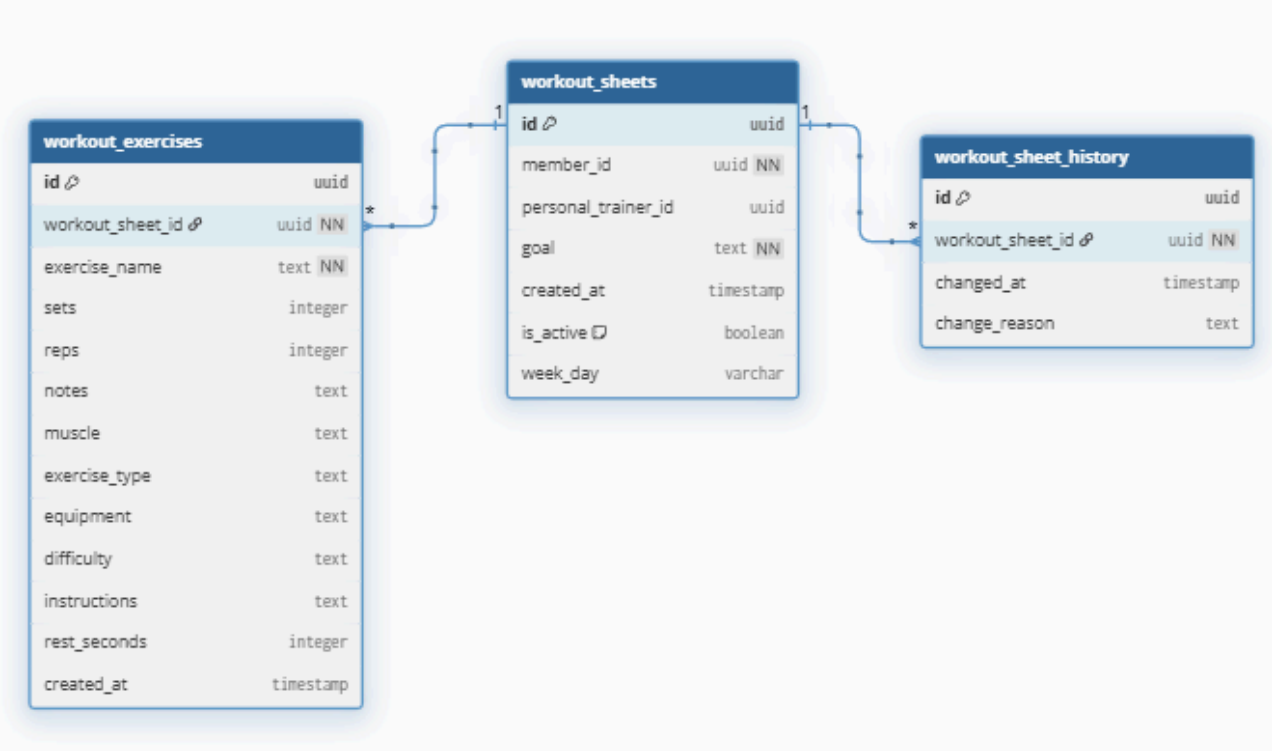


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta implementação refere-se apenas ao backend da aplicação.

Estória 17: Permitir consulta de dados pelo aluno
Como aluno, quero consultar meus dados no painel do aluno, para visualizar minha assinatura, pagamentos e fichas de treino.

Especificação do Requisito

O sistema oferece autosserviço para consulta no painel do aluno, permitindo que os alunos visualizem suas próprias informações cadastrais, status da assinatura, pagamentos e fichas de treino ativas.

O aluno deve visualizar apenas seus próprios dados.

O aluno deve consultar sua assinatura ativa.

O aluno deve visualizar o status de sua assinatura.

O aluno deve consultar seus pagamentos registrados.

O aluno deve consultar suas fichas de treino.

O aluno deve visualizar os exercícios prescritos.

O aluno deve poder alternar a visualização dos treinos por dia da semana.

O aluno não deve alterar dados administrativos.

O aluno não deve alterar pagamentos.

O aluno não deve alterar fichas de treino.

O acesso deve ser permitido apenas ao perfil OPERATIONAL.

Diagrama de Classe do Contexto

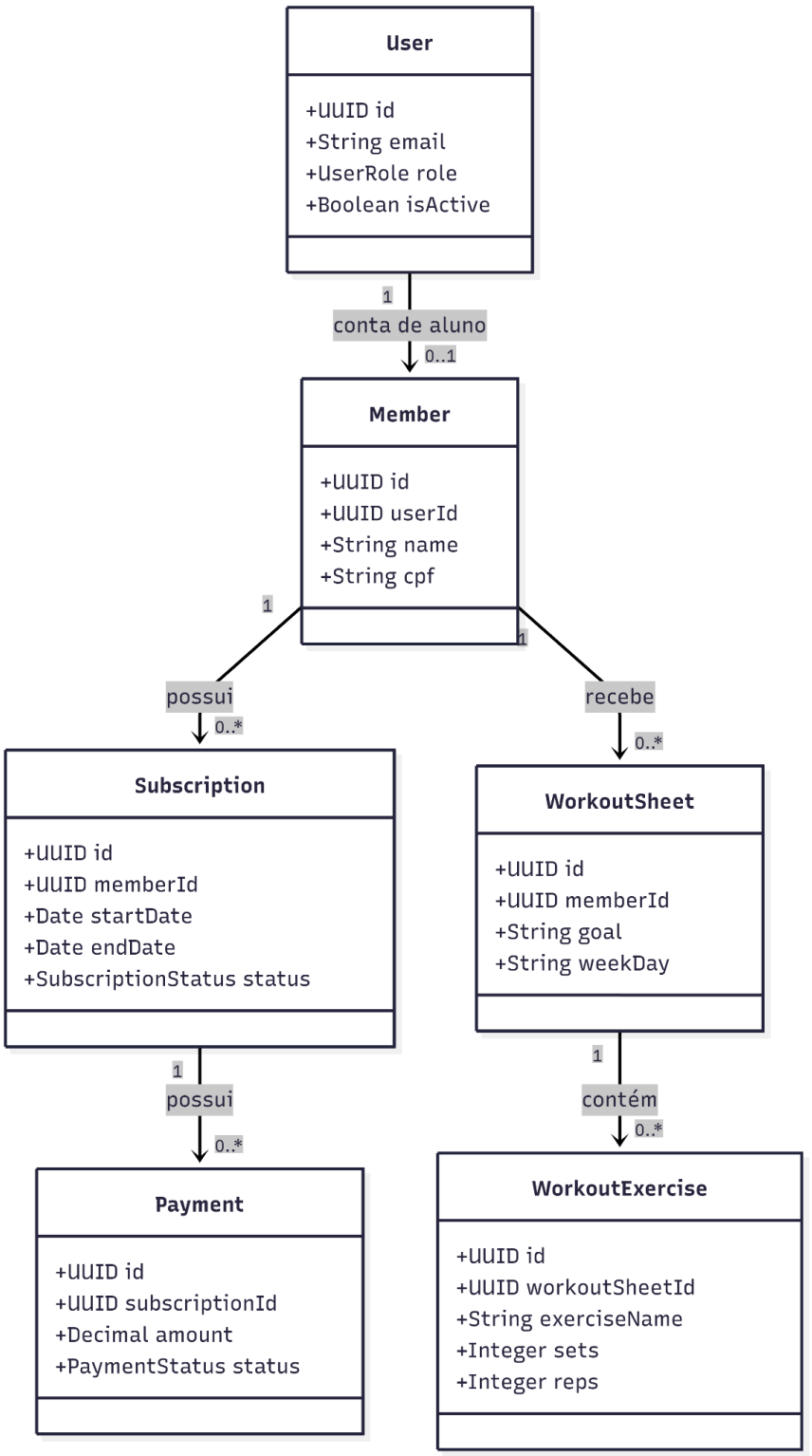
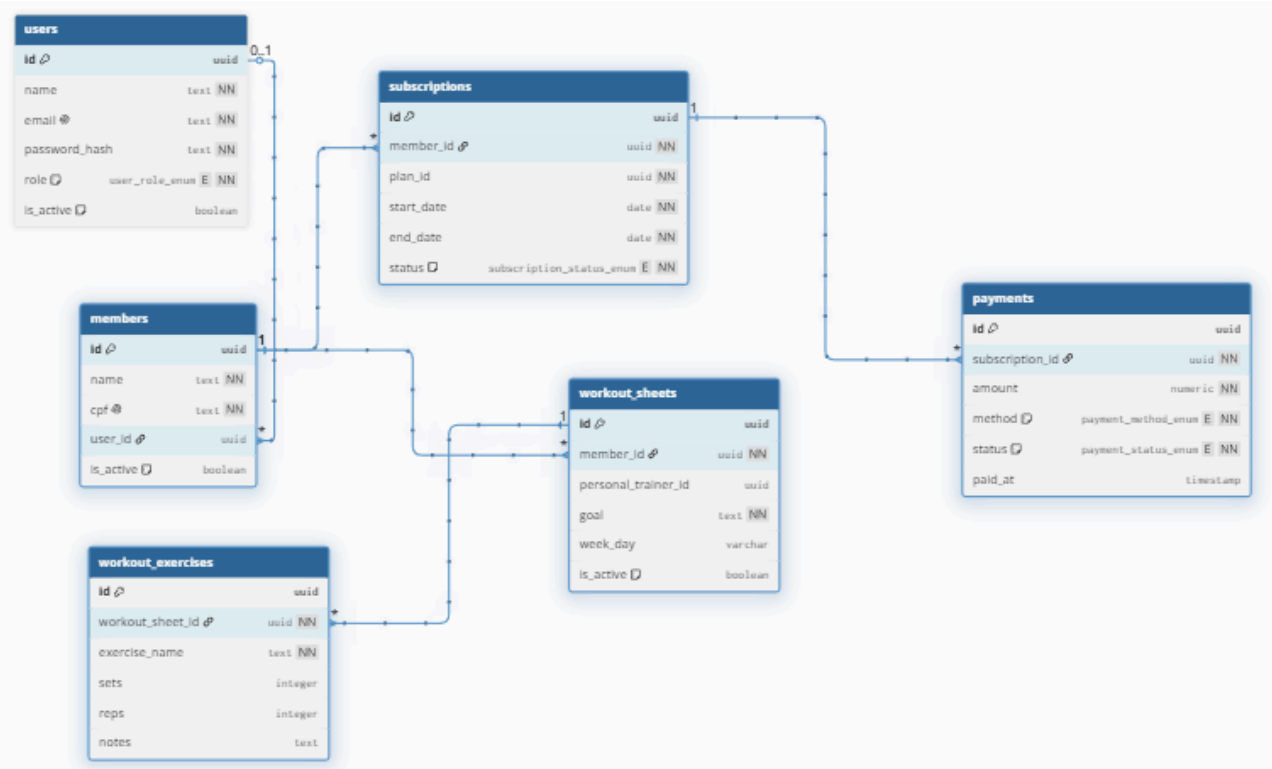


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
Esta funcionalidade possui interface no painel do aluno, permitindo consulta de assinatura, pagamentos e fichas de treino em modo somente leitura.

Estória 18: Controlar permissões por perfil de acesso
Como administrador do sistema, quero controlar permissões por perfil de acesso, para garantir que cada usuário utilize apenas as funcionalidades permitidas.

Especificação do Requisito

O sistema deve controlar acesso por perfil de usuário.

Os perfis disponíveis devem ser:

- ADMIN;
- PERSONAL_TRAINER;
- OPERATIONAL.

O perfil ADMIN deve acessar funções administrativas, financeiras e relatórios.

O perfil PERSONAL_TRAINER deve acessar alunos vinculados e fichas de treino.

O perfil OPERATIONAL deve acessar apenas dados próprios.

O sistema deve proteger rotas com autenticação JWT.

O sistema deve bloquear requisições sem token válido.

O sistema deve bloquear usuários inativos.

O sistema deve impedir acesso a funcionalidades fora do perfil.

O sistema deve validar permissões no backend.

O sistema deve direcionar o usuário para a área correta após login.

Diagrama de Classe do Contexto

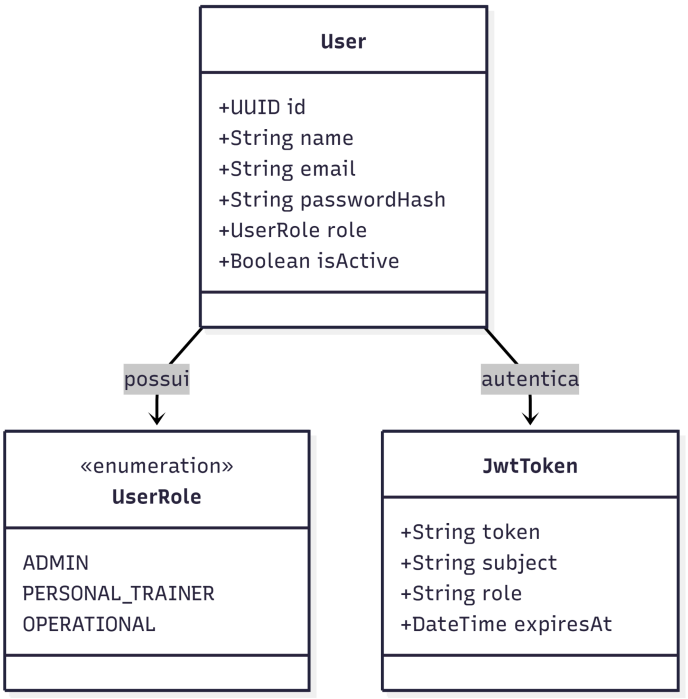
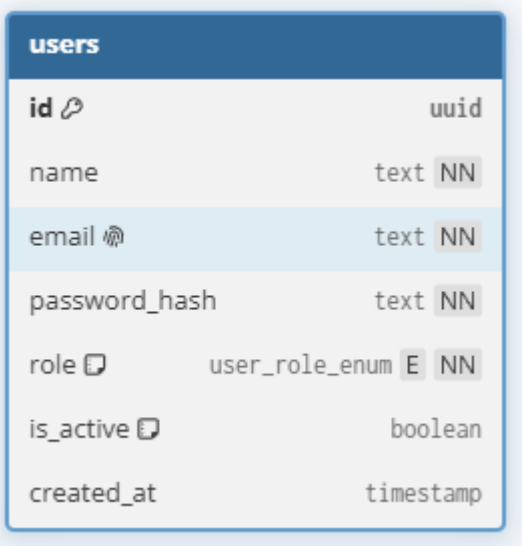


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Não se aplica, pois esta estória de usuário refere-se ao controle de permissões e regras de acesso na camada de backend da aplicação, não envolvendo diretamente a construção de interfaces de usuário.

Estória 19:Integrar pagamento via gateway online
Como aluno, quero realizar pagamentos online pelo sistema, para pagar meu plano de forma prática e integrada.

Especificação do Requisito

A funcionalidade foi implementada tecnicamente em ambiente local utilizando mocks e simulações, integrada ao Mercado Pago para processamento de transações via Pix e cartões.

O sistema deverá permitir pagamento online de planos.

O aluno deverá selecionar uma assinatura ou plano a ser pago.

O sistema deverá enviar a solicitação de pagamento para um gateway externo.

O gateway poderá ser Mercado Pago, Stripe, Pix automático ou outro provedor equivalente.

O sistema deverá armazenar dados da transação online.

O sistema deverá registrar identificador da transação.

O sistema deverá registrar valor, status e data da solicitação.

O pagamento online não substitui o registro manual existente na versão atual.

A funcionalidade deverá integrar-se às entidades de assinatura e pagamento.

A integração foi validada em ambiente de testes locais utilizando mocks e simulações, sem homologação em produção ou sandbox externo real.

Diagrama de Classe do Contexto

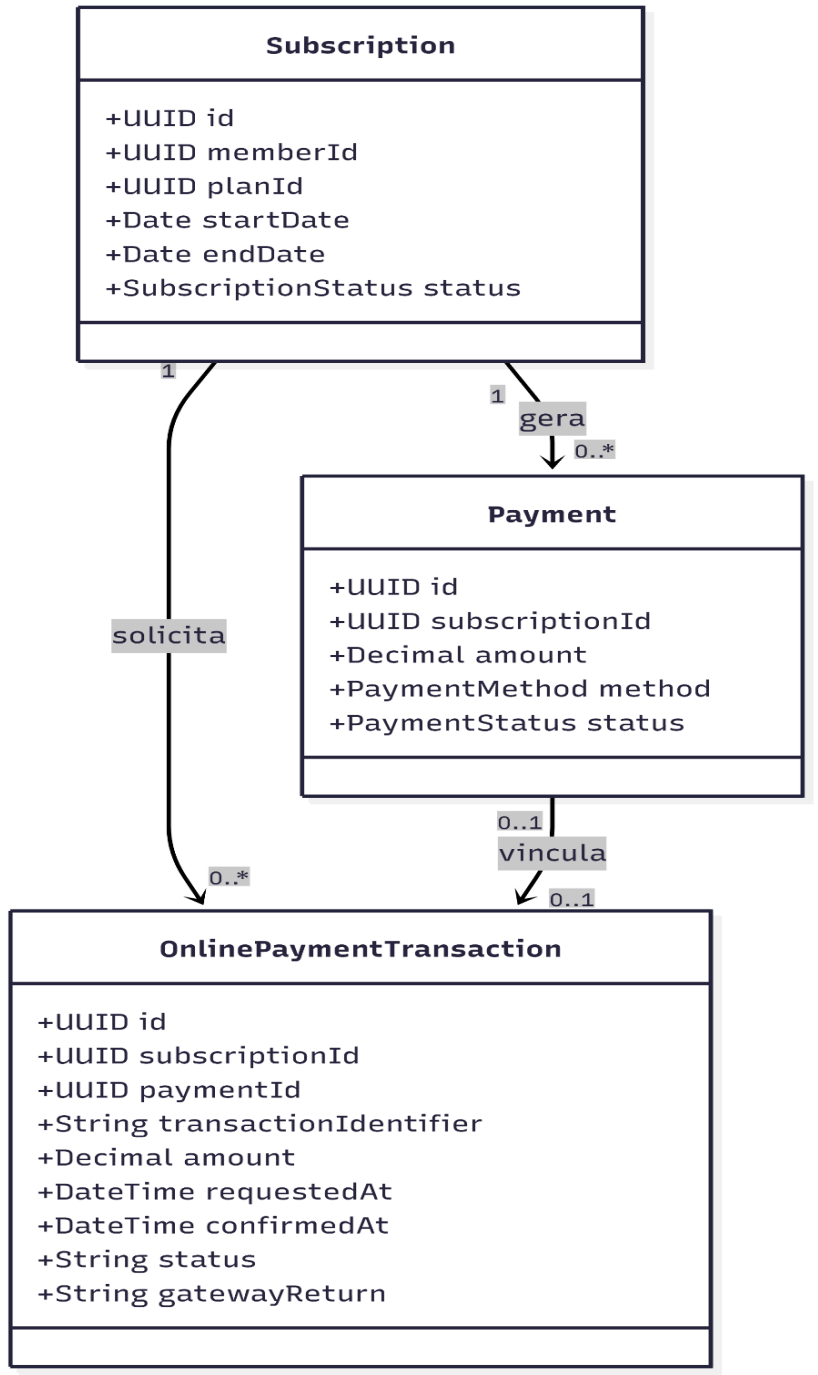
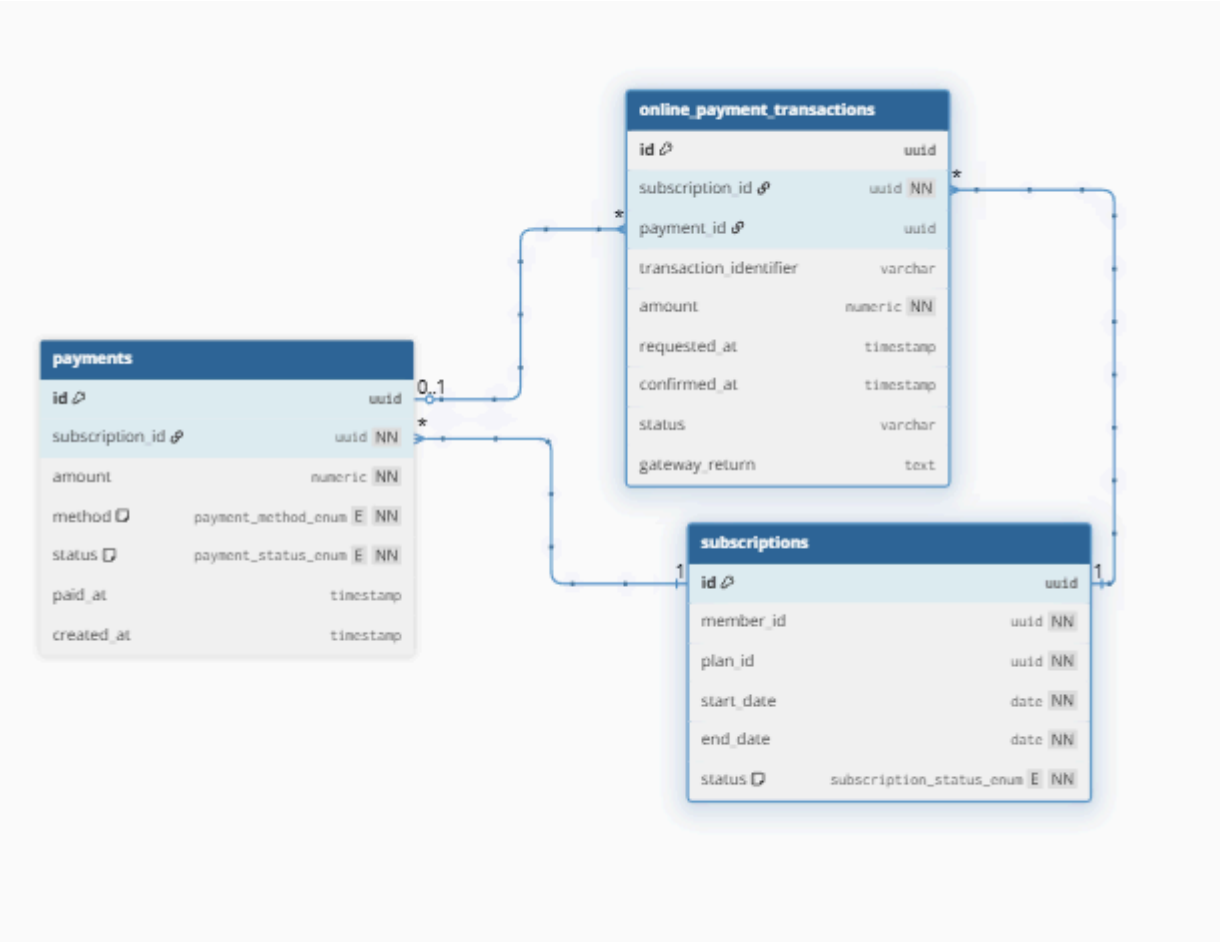


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
Funcionalidade prevista para evolução futura. O protótipo poderá contemplar seleção do plano, confirmação de pagamento online e retorno do status da transação.

Estória 20: Processar confirmação automática de pagamento online
Como sistema, quero processar automaticamente a confirmação de pagamento online, para atualizar o pagamento e a assinatura sem intervenção manual.

Especificação do Requisito

A confirmação automática de pagamentos online via webhooks/retorno do gateway foi implementada tecnicamente e testada em ambiente local utilizando mocks e simulações, sem homologação em produção ou sandbox externo real.

O sistema deverá receber retorno automático do gateway de pagamento.

O retorno poderá ocorrer por webhook ou consulta à API do provedor.

O sistema deverá localizar a transação correspondente.

O sistema deverá validar se a transação foi aprovada.

Em caso de pagamento aprovado, o sistema deverá atualizar o status do pagamento.

Em caso de pagamento aprovado, o sistema deverá atualizar a assinatura correspondente.

Em caso de falha, o sistema deverá registrar o retorno do gateway.

O sistema deverá manter histórico do status da transação.

A confirmação automática encontra-se plenamente validada em ambiente local através de simulações e mocks do Mercado Pago.

O recurso deverá reduzir a necessidade de baixa manual de pagamentos.

Diagrama de Classe do Contexto

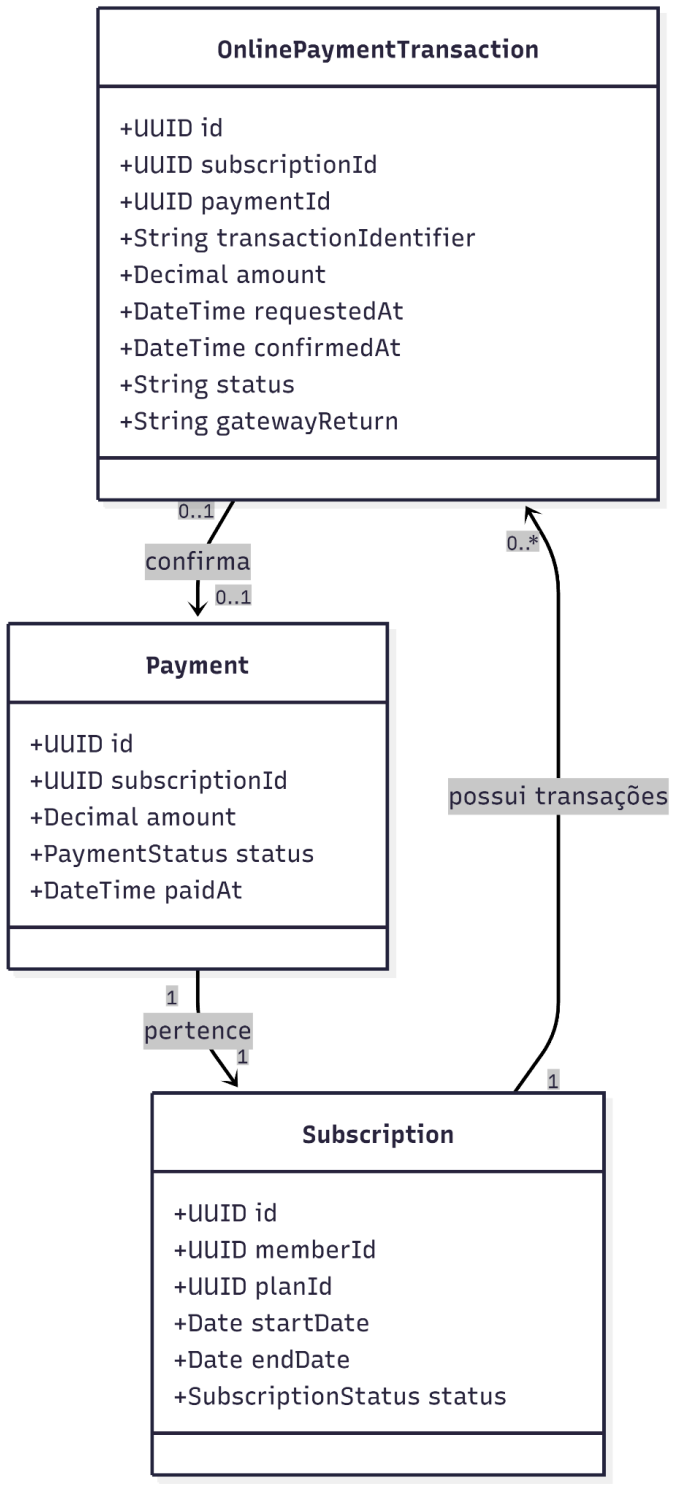
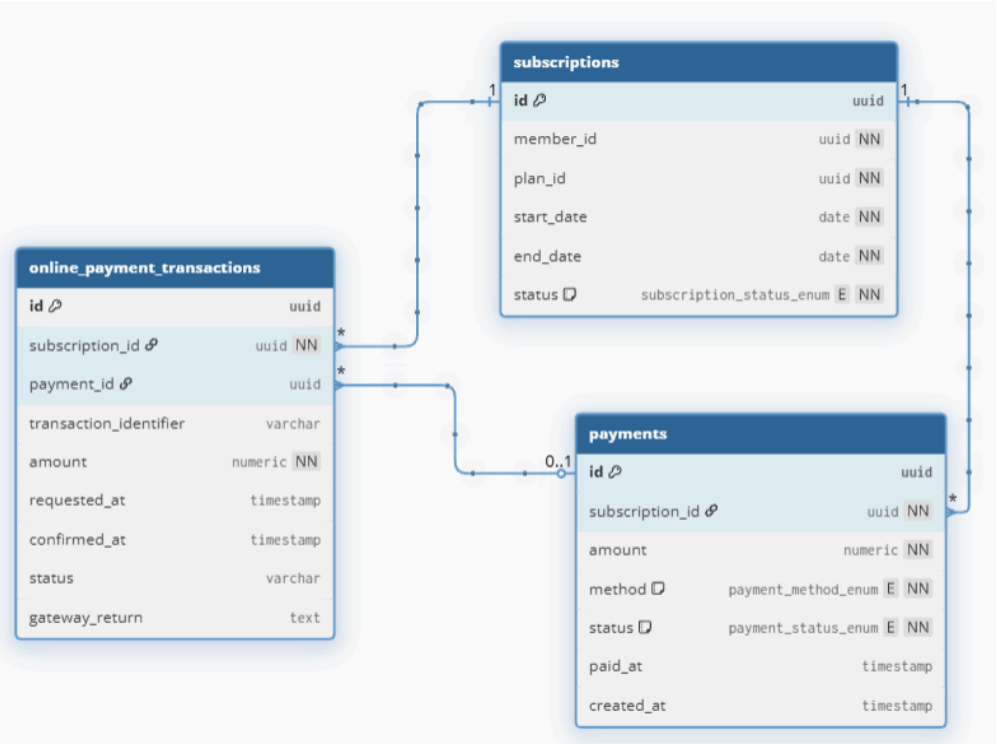


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

Funcionalidade prevista para evolução futura. Não se aplica à versão atual, pois depende de integração real com gateway de pagamento.

Estória 21: Gerar relatório de novos alunos por período
Como administrador do sistema, quero gerar relatório de novos alunos por período, para acompanhar o crescimento da academia.

Especificação do Requisito

O sistema deve permitir gerar relatório de novos alunos.

O relatório deve permitir filtro por data inicial e data final.

O sistema deve listar os alunos cadastrados no período.

O relatório deve apresentar o nome do aluno.

O relatório deve apresentar o CPF do aluno.

O relatório deve apresentar a data de cadastro.

O sistema deve apresentar o total de novos alunos no período.

Apenas usuários ADMIN devem acessar o relatório.

O relatório deve auxiliar a gestão no acompanhamento de aquisição de alunos.

Diagrama de Classe do Contexto

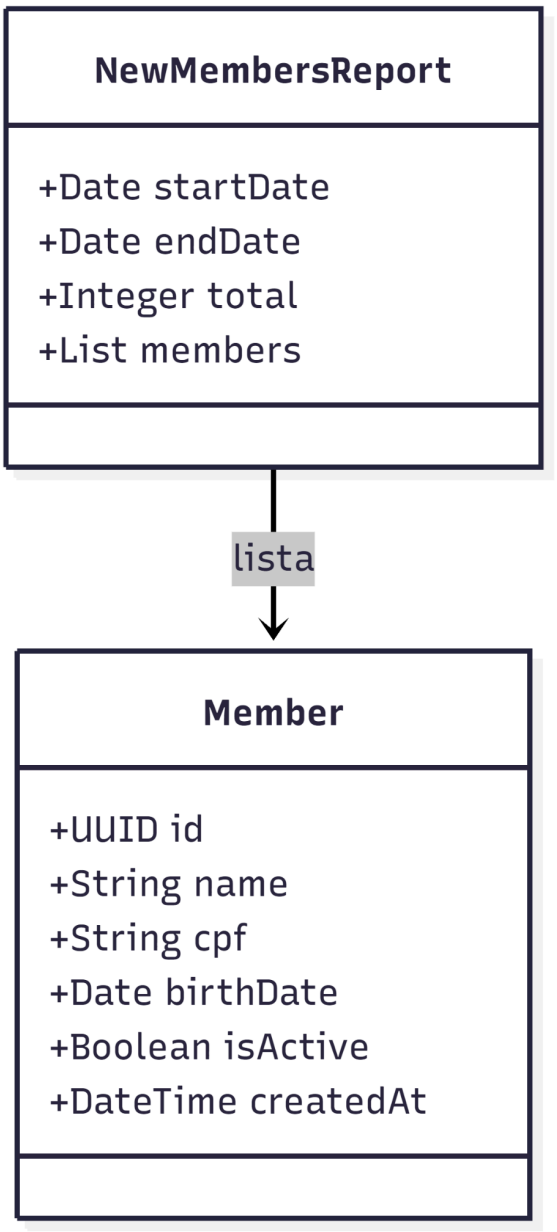
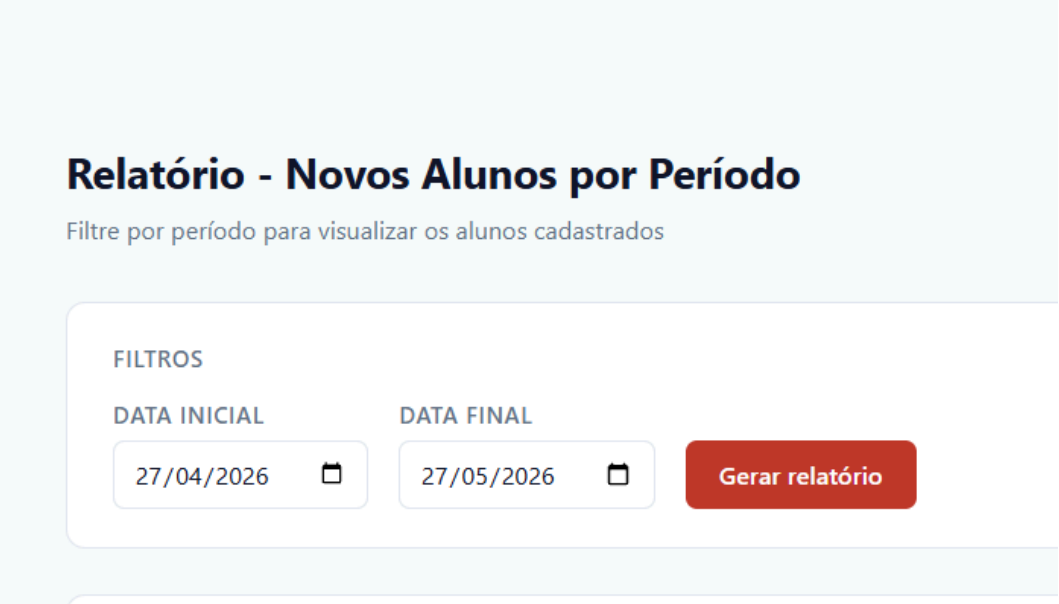


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
A interface deve permitir informar o período desejado e visualizar a listagem de alunos cadastrados, bem como o total de novos cadastros.



Estória 22: Gerar relatório de cancelamentos
Como administrador do sistema, quero gerar relatório de cancelamentos, para acompanhar os desligamentos e motivos de evasão dos alunos.

Especificação do Requisito

O sistema deve permitir gerar relatório de cancelamentos.

O relatório deve permitir filtro por período.

O sistema deve listar os alunos cancelados.

O relatório deve apresentar a data do cancelamento.

O relatório deve apresentar o motivo informado.

O relatório deve apresentar os dados básicos do aluno.

O relatório deve apoiar a análise de evasão.

Apenas usuários ADMIN devem acessar o relatório.

Os dados devem ser obtidos a partir dos registros de cancelamento.

Diagrama de Classe do Contexto

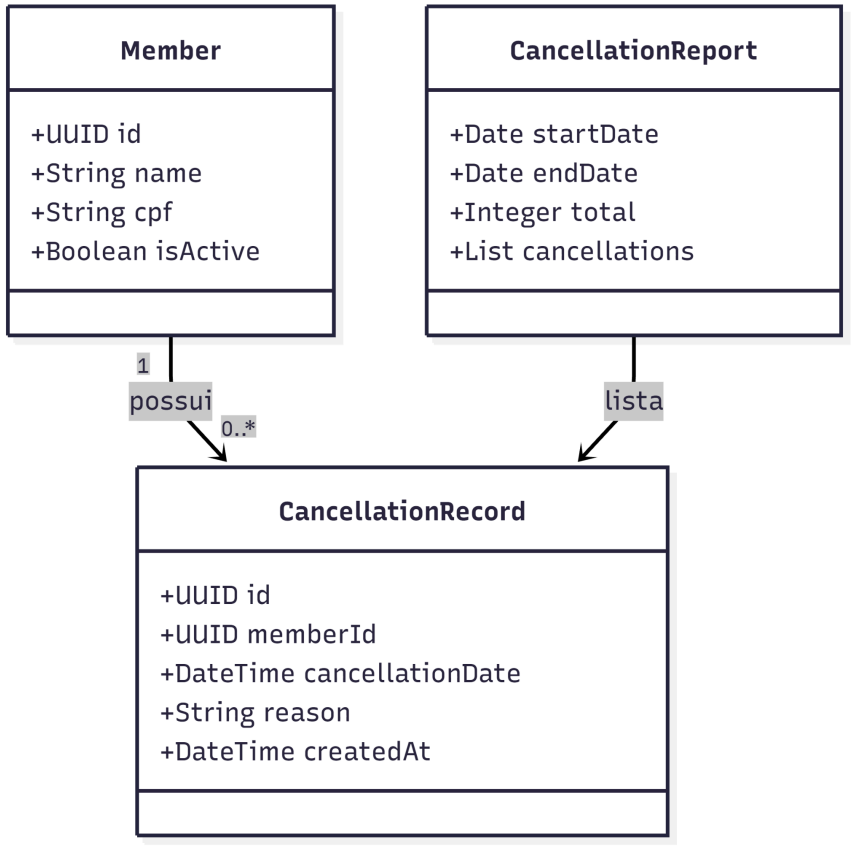
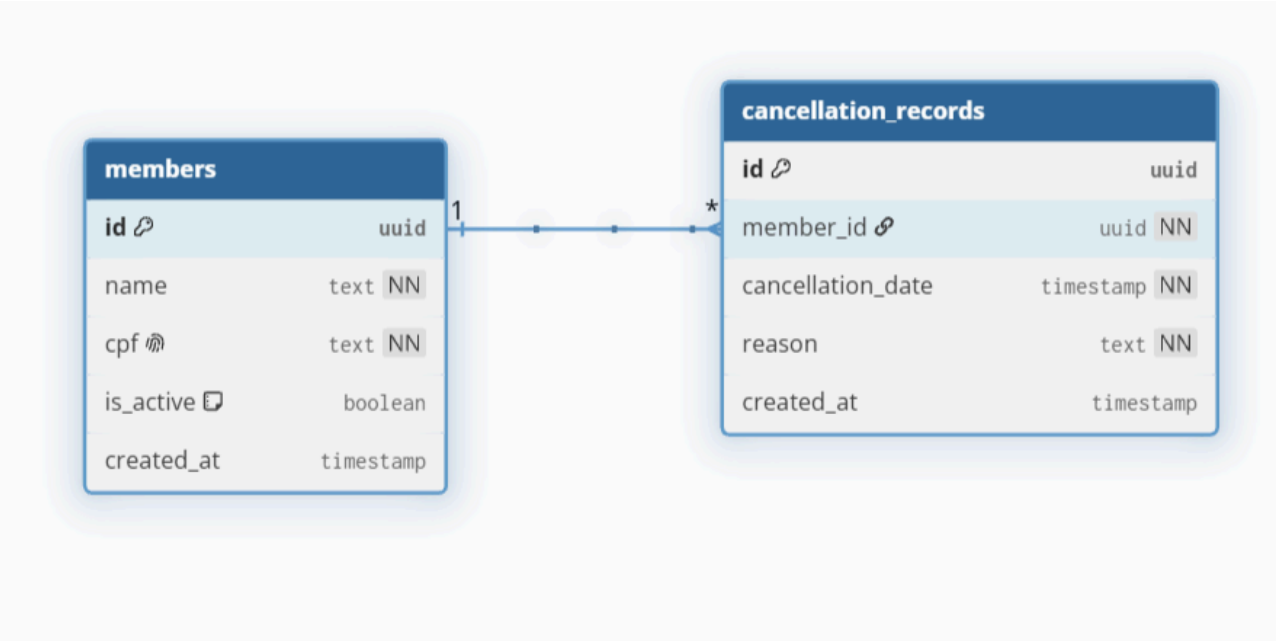
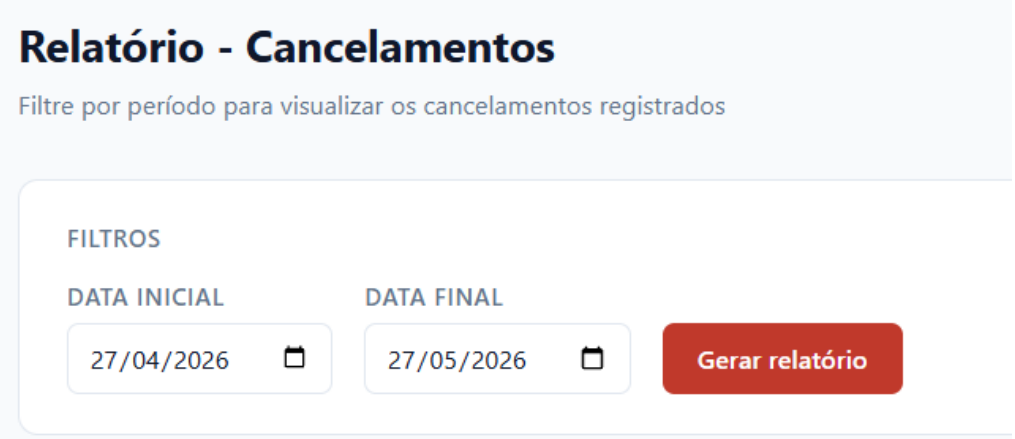


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
A interface deve permitir selecionar um período e visualizar os alunos cancelados, suas datas de cancelamento e os motivos registrados.



Estória 23: Gerar relatório de alunos ativos e inativos
Como administrador do sistema, quero gerar relatório de alunos ativos e inativos, para acompanhar a situação cadastral dos alunos da academia.

Especificação do Requisito

O sistema deve permitir gerar relatório de alunos ativos e inativos.

O relatório deve considerar o campo de status do aluno.

O sistema deve listar alunos ativos.

O sistema deve listar alunos inativos.

O relatório deve apresentar quantitativo de ativos.

O relatório deve apresentar quantitativo de inativos.

O relatório deve auxiliar o controle administrativo.

Apenas usuários ADMIN devem acessar o relatório.

A inativação deve ser baseada em soft delete.

Diagrama de Classe do Contexto

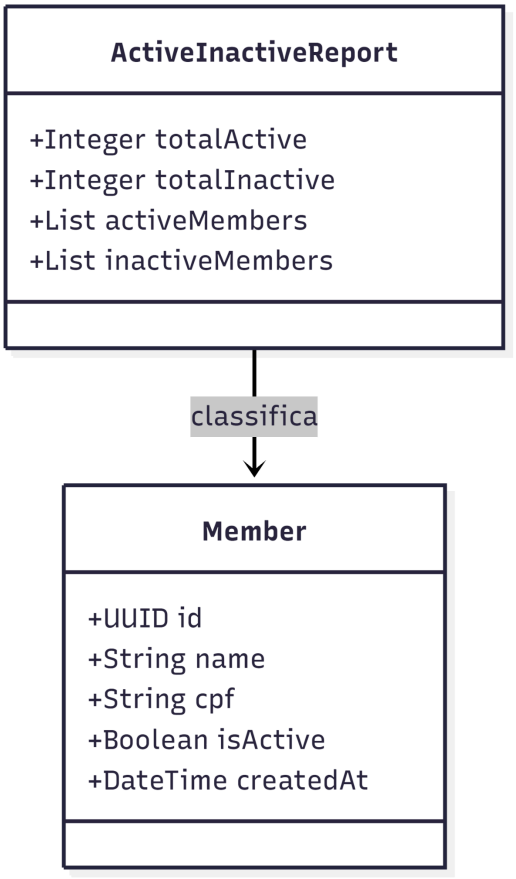
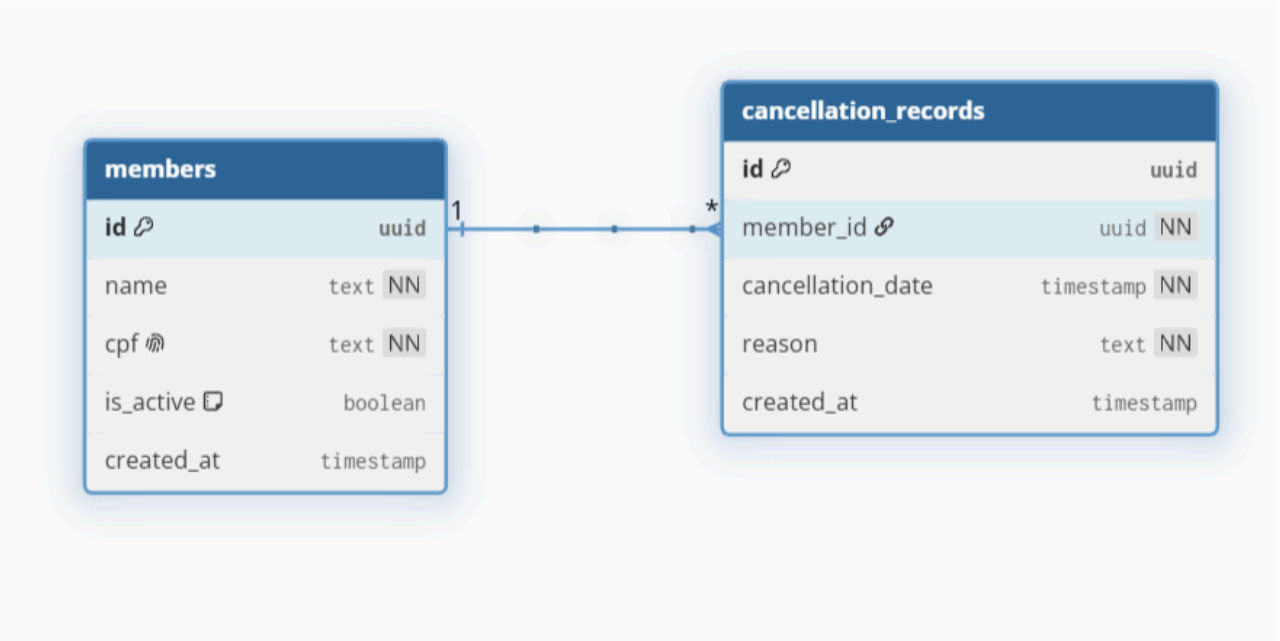
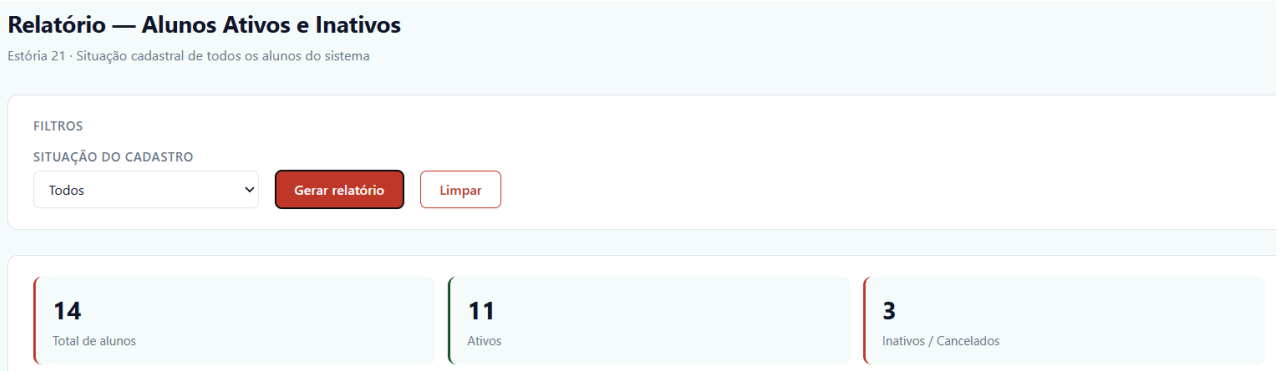


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
A interface apresenta filtros por situação cadastral (Todos, Ativos ou Inativos) e exibe cards com o quantitativo total de alunos, total de ativos e total de inativos/cancelados no sistema.



Estória 24: Gerar relatório de alunos inadimplentes
Como administrador do sistema, quero gerar relatório de alunos inadimplentes, para acompanhar pendências financeiras e apoiar ações de cobrança.

Especificação do Requisito

O sistema deve permitir gerar relatório de alunos inadimplentes.

O relatório deve considerar pagamentos pendentes ou atrasados.

O relatório deve considerar assinaturas vencidas.

O sistema deve listar os alunos inadimplentes.

O relatório deve apresentar dados do aluno.

O relatório deve apresentar informações da assinatura.

O relatório deve apresentar informações do pagamento.

O relatório deve apoiar ações administrativas de cobrança.

O sistema não deve aplicar juros ou multa automaticamente.

Apenas usuários ADMIN devem acessar o relatório.

Diagrama de Classe do Contexto

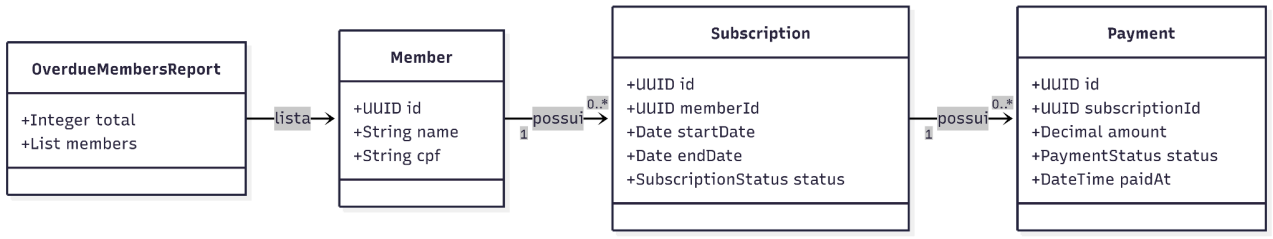
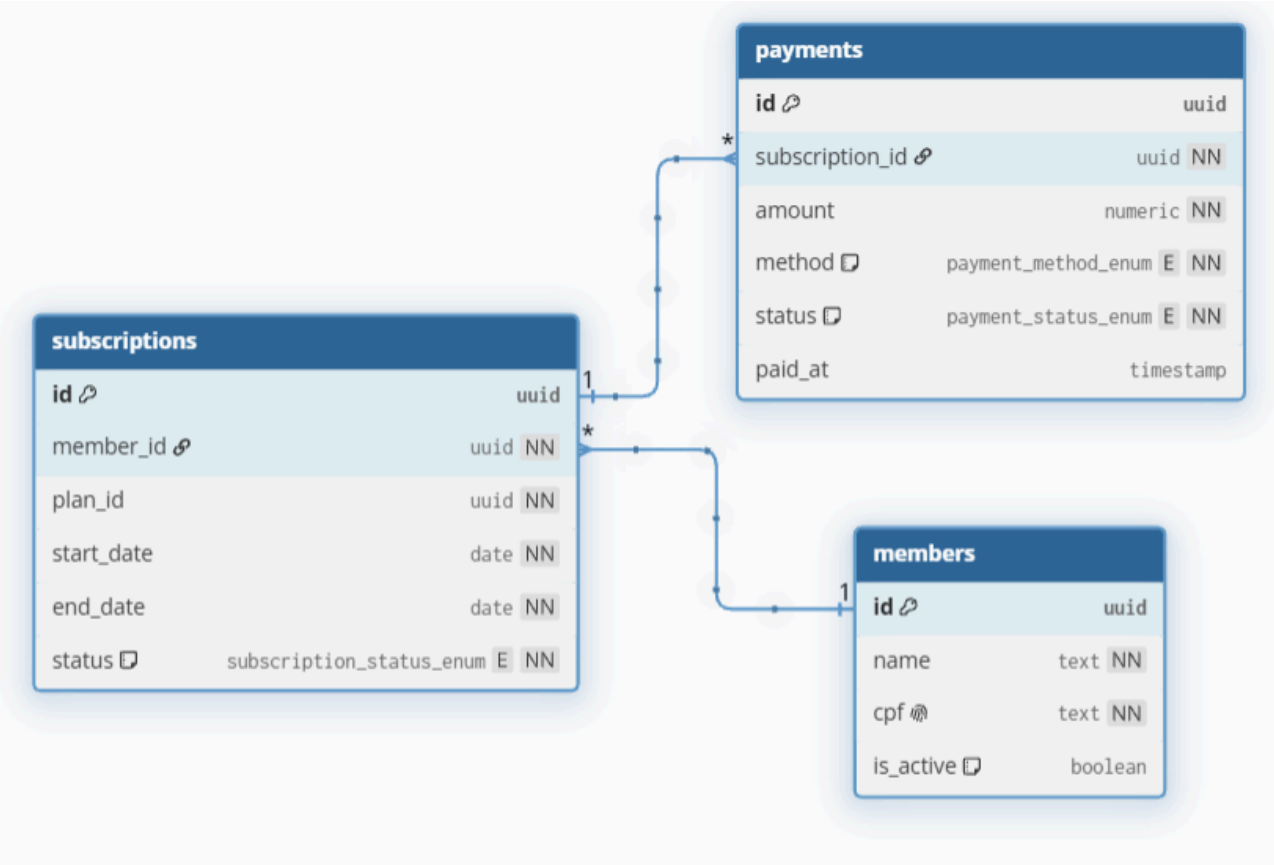
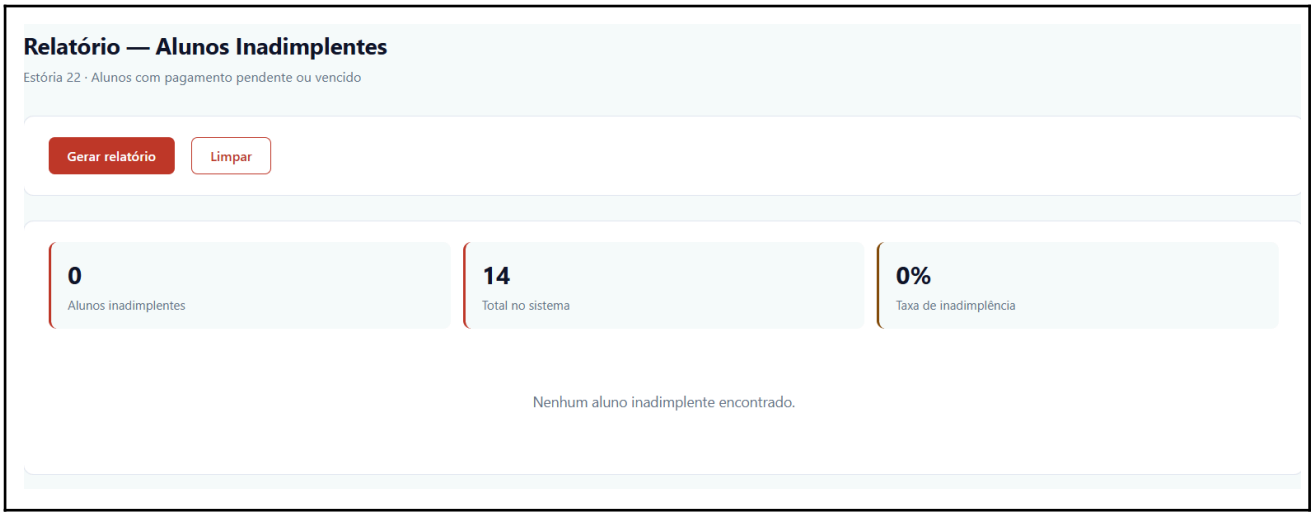


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)
A interface deve permitir visualizar os alunos inadimplentes, seus planos, status financeiro e informações relevantes para cobrança.



Estória 25: Apresentar dashboards administrativos e indicadores financeiros
Como administrador do sistema, quero visualizar dashboards administrativos e indicadores financeiros, para acompanhar a situação geral da academia.

Especificação do Requisito

O sistema deve apresentar dashboard administrativo.

O dashboard deve exibir indicadores gerais da academia.

O sistema deve apresentar total de alunos cadastrados.

O sistema deve apresentar quantidade de alunos ativos.

O sistema deve apresentar quantidade de alunos inativos.

O sistema deve apresentar alunos inadimplentes.

O sistema deve apresentar informações de pagamentos.

O sistema deve apresentar informações de cancelamentos.

O sistema deve apoiar a tomada de decisão.

Apenas usuários ADMIN devem acessar os dashboards.

Os indicadores devem ser calculados a partir dos dados cadastrados no sistema.

Diagrama de Classe do Contexto

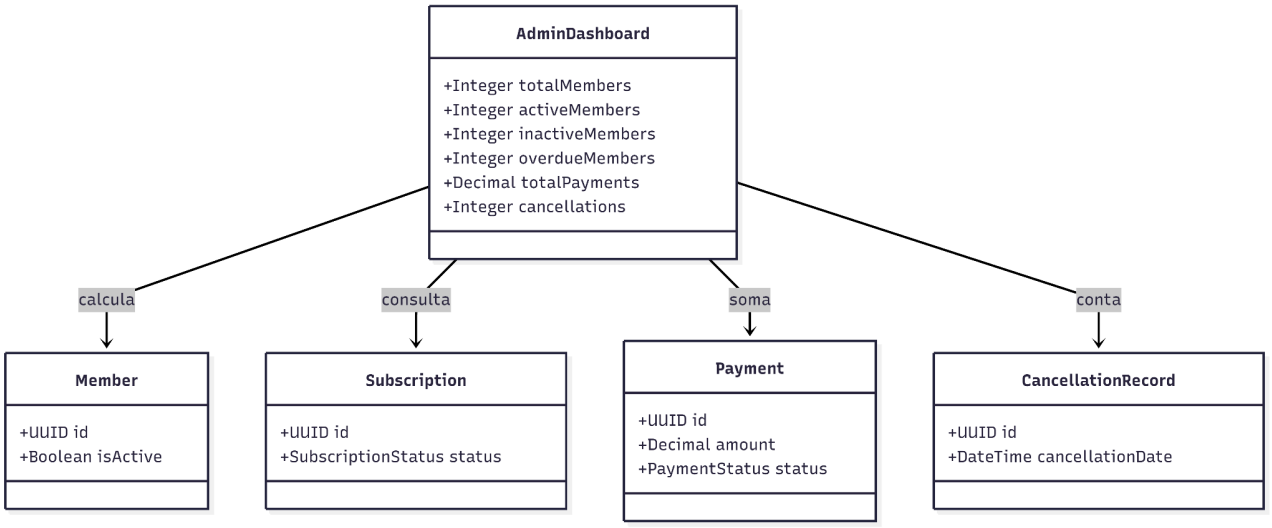
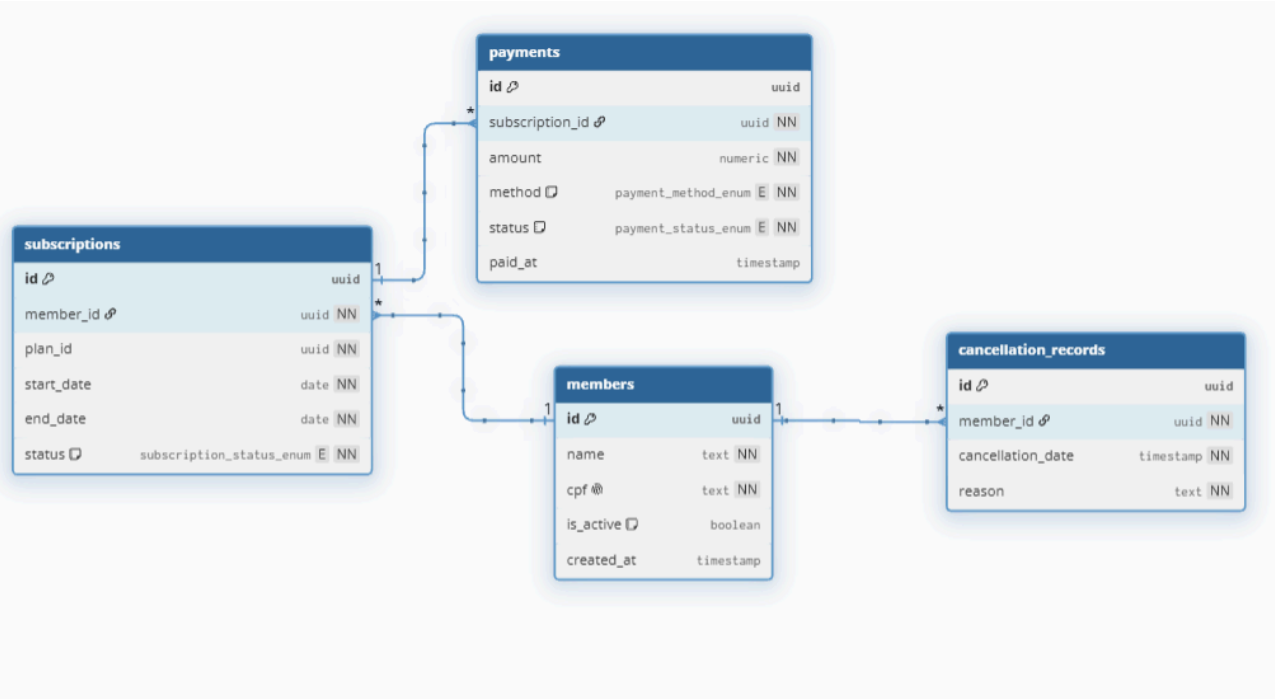


Diagrama de ER do Contexto



Detalhamento com protótipo (Interface de Usuário)

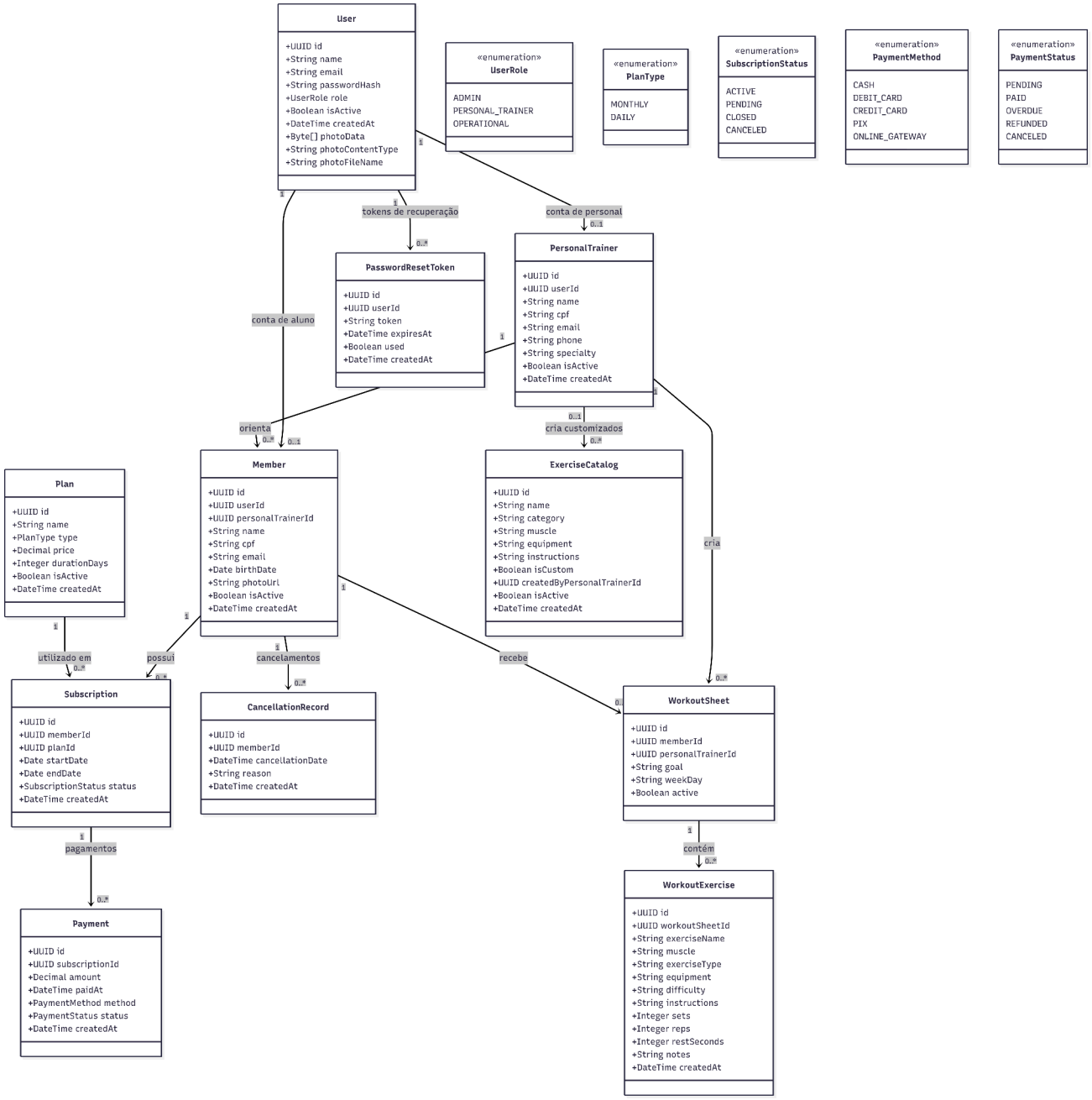
A interface deve apresentar cards, indicadores e visualizações gerenciais para apoiar a administração da academia.

3. PROJETO DE SOFTWARE

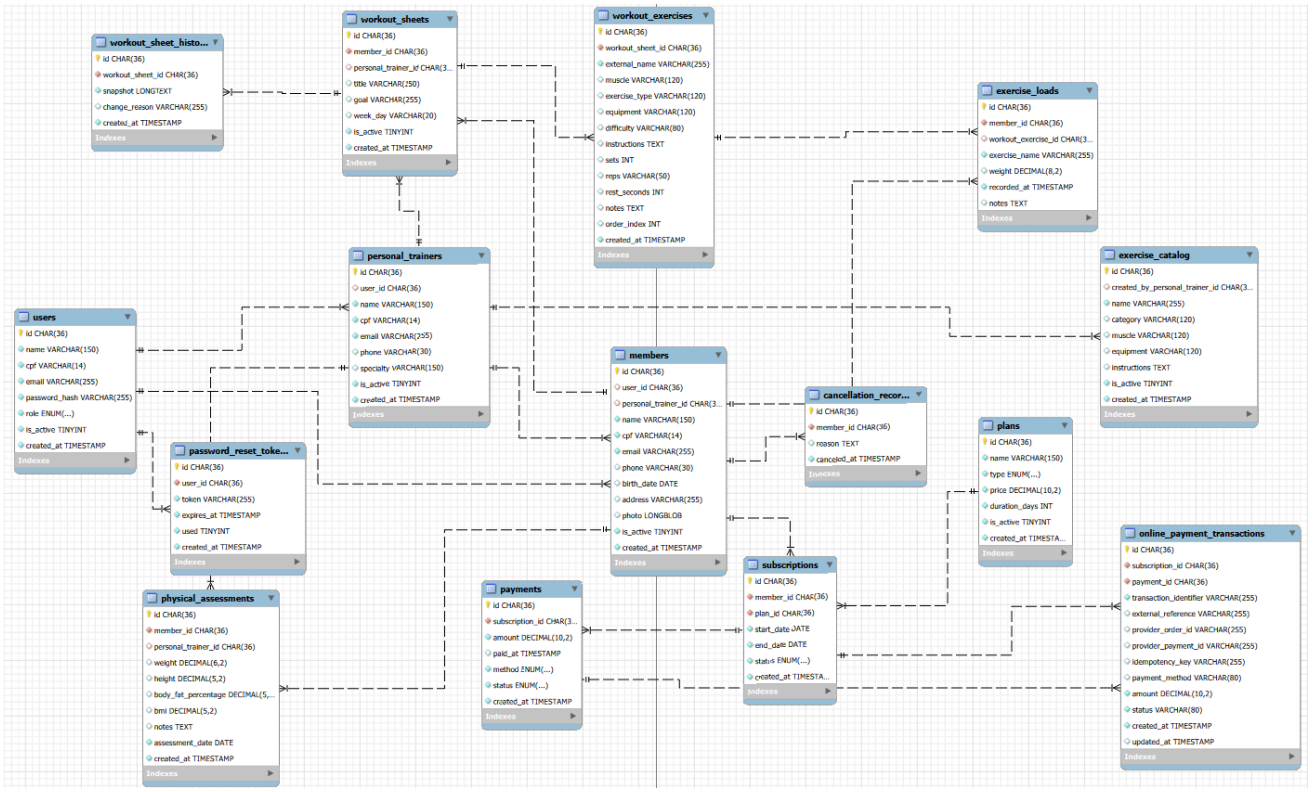
Nesta seção, serão apresentados dois dos principais artefatos utilizados no desenvolvimento de sistemas: o **Diagrama de Classes** e o **Diagrama Entidade-Relacionamento (DER)**. Esses diagramas são fundamentais para representar visualmente a estrutura e o funcionamento do sistema, garantindo uma compreensão clara da lógica de negócio e da arquitetura do software.

3.1 Diagrama de Classes Final

Observação: Os diagramas apresentados nesta seção refletem a modelagem conceitual e estrutural inicial do projeto. Estruturas adicionais do backend, como a tabela de histórico de alterações de fichas de treino (workout_sheet_history) e as transações de pagamento online, representam evoluções posteriores do sistema.



3.2 Diagrama Entidade-Relacionamento Final



Observação: O Diagrama Entidade-Relacionamento abaixo representa o modelo relacional inicial da aplicação. As tabelas de pagamentos online (online_payment_transactions) e histórico de fichas (workout_sheet_history) correspondem a aprimoramentos técnicos posteriores na camada de persistência.

4. PROCEDIMENTOS DE INSTALAÇÃO

4.1 Requisitos do Sistema

Para execução do sistema Lion Fitness, o ambiente deve possuir os seguintes requisitos mínimos:

- Computador com acesso à internet;
- Sistema operacional Windows, Linux ou macOS;
- Navegador web atualizado (Google Chrome, Microsoft Edge, Mozilla Firefox ou equivalente);
- Ambiente de desenvolvimento, como IntelliJ IDEA, Visual Studio Code ou equivalente;
- Java JDK 21 instalado;
- Framework Spring Boot versão 3.5.7 (Backend);
- Apache Maven instalado ou utilização do Maven Wrapper (./mvnw) incluído no projeto;
- Node.js versão 20.9.0 ou superior instalado;
- Next.js versão 16 (Frontend);
- Gerenciador de pacotes npm ou yarn;
- Banco de dados PostgreSQL (hospedado no Supabase) com suporte ao tipo BYTEA na tabela users (campo photo_data);
- Git instalado para clonagem do repositório;
- Variáveis de ambiente exatas configuradas para backend e frontend.
- Sistema operacional Windows, Linux ou macOS;
- Navegador web atualizado (Google Chrome, Microsoft Edge, Mozilla Firefox ou equivalente);
- Ambiente de desenvolvimento, como IntelliJ IDEA, Visual Studio Code ou equivalente;
- Java JDK 21 instalado;
- Apache Maven instalado (ou utilização do Maven Wrapper do projeto);
- Node.js versão 20.9.0 ou superior instalado;
- Gerenciador de pacotes npm ou yarn;
- Conta no Supabase para utilização do banco de dados PostgreSQL;
- Git instalado para clonagem do repositório;
- Variáveis de ambiente configuradas para backend e frontend.

4.2 Etapas de Instalação

4.2.1 Obtenção do Projeto

O código-fonte do sistema deve ser obtido através do repositório oficial do projeto utilizando o comando:

- git clone https://github.com/olucasortiz/estagio-supervisionado-lionfitess.git

---

4.2.2 Acesso ao Diretório do Projeto

Após a clonagem do repositório, deve-se acessar o diretório principal do projeto:

- cd estagio-supervisionado-lionfitess

4.2.3 Estrutura do Projeto

O sistema encontra-se dividido em dois módulos principais:

- Backend:
    - API REST desenvolvida em Spring Boot;
    - Responsável pelas regras de negócio, autenticação JWT, persistência de dados e segurança.
- Frontend:
    - Aplicação web desenvolvida em Next.js e React;
    - Responsável pela interface do usuário, dashboards e interação com a API.

4.2.4 Configuração do Banco de Dados

O sistema utiliza PostgreSQL hospedado no Supabase.

Para configuração do banco de dados, devem ser realizados os seguintes passos:

1. Criar um projeto no Supabase;
2. Executar os scripts SQL responsáveis pela criação das tabelas e relacionamentos;
3. Obter as credenciais de conexão:
    - Host;
    - Database;
    - Username;
    - Password;
4. Configurar as variáveis de ambiente do backend.

Exemplo de configuração das variáveis de ambiente no arquivo `application.properties` ou ambiente de execução:

- SPRING_DATASOURCE_URL=jdbc:postgresql://aws-1-us-east-2.pooler.supabase.com:5432/postgres?sslmode=require
- SPRING_DATASOURCE_USERNAME=postgres.seu_project_ref
- SPRING_DATASOURCE_PASSWORD=SUA_SENHA
- MERCADOPAGO_ACCESS_TOKEN=SEU_ACCESS_TOKEN_LOCAL_MOCK
- spring.datasource.username=postgres.seu_project_ref
- spring.datasource.password=SUA_SENHA
- spring.datasource.driver-class-name=org.postgresql.Driver
-
- spring.datasource.hikari.maximum-pool-size=5
- spring.datasource.hikari.minimum-idle=1
- spring.datasource.hikari.connection-timeout=20000
-
- spring.jpa.hibernate.ddl-auto=none
- spring.jpa.show-sql=false

Também devem ser configuradas as variáveis relacionadas ao JWT e envio de e-mails:

- security.jwt.secret=SUA_CHAVE_SECRETA
- security.jwt.expiration-seconds=86400
-
- spring.mail.host=smtp.gmail.com
- spring.mail.port=587
- spring.mail.username=SEU_EMAIL

- spring.mail.password=SUA_SENHA_DE_APLICATIVO
- spring.mail.properties.mail.smtp.auth=true
- spring.mail.properties.mail.smtp.starttls.enable=true

4.2.5 Execução do Backend (Spring Boot)

Acesse a pasta do backend:

- cd estagio-supervisionado-lionfitness

Execute a aplicação utilizando o Maven Wrapper do projeto:

- ./mvnw spring-boot:run

Ou, caso utilize Maven Wrapper:

- ./mvnw spring-boot:run

Após a inicialização, o backend estará disponível em:

- http://localhost:8080

4.2.6 Instalação das Dependências do Frontend

Acesse a pasta do frontend:

- cd estagio-supervisionado-lionfitess-frontend

Instale as dependências do projeto:

- npm install

4.2.7 Configuração das Variáveis de Ambiente do Frontend

Criar um arquivo `.env.local` na raiz do frontend contendo:

- NEXT_PUBLIC_API_URL=http://localhost:8080

Caso o frontend esteja utilizando ambiente de produção, deve-se informar a URL pública da API backend.

4.2.8 Execução do Frontend (Next.js)

Execute o frontend utilizando:

- npm run dev

Após a inicialização, o frontend estará disponível em:

- http://localhost:3000

4.2.9 Acesso à Aplicação

Após a execução dos módulos:

- Frontend:
  http://localhost:3000
-
- Backend:
  http://localhost:8080
- Sistema rodando na URL: https://estagio-supervisionado-monorepo.vercel.app/

4.3 Considerações Finais

Antes da utilização do sistema, recomenda-se verificar:

- Se o backend está executando corretamente na porta 8080;
- Se o frontend está executando corretamente na porta 3000;
- Se o banco de dados Supabase encontra-se acessível;
- Se as credenciais de conexão estão corretas;
- Se as variáveis de ambiente foram configuradas corretamente;
- Se não existem conflitos de portas locais;
- Se o CORS está devidamente configurado no backend;
- Se o token JWT está sendo gerado corretamente após autenticação;
- Se o envio de e-mails está funcionando corretamente;
- Se o frontend consegue consumir a API backend sem erros de comunicação.

O sistema também pode ser implantado em ambiente de produção utilizando plataformas de hospedagem como Render, Railway ou Vercel.

---

5. REFERÊNCIAS

Documentos Utilizados

- Documento de Levantamento de Requisitos do sistema Lion Fitness;
- Modelo acadêmico de Engenharia de Requisitos de Software (ERS Ágil);
- Diagramas MER, DER e UML desenvolvidos durante a modelagem do sistema;
- Relatórios de modelagem de dados e arquitetura do projeto;
- Estrutura real do banco de dados PostgreSQL extraída do Supabase;
- Regras de negócio levantadas durante o desenvolvimento do sistema.

Ferramentas Utilizadas

- dbdiagram.io — modelagem e construção do Diagrama Entidade-Relacionamento (DER);
- Mermaid — geração de diagramas UML e diagramas conceituais;
- Supabase (PostgreSQL) — persistência e gerenciamento de dados;
- Java 21 — desenvolvimento backend;
- Spring Boot 3.x — desenvolvimento da API REST;
- Spring Security + JWT — autenticação e controle de acesso;
- React — desenvolvimento da interface web;
- Next.js — estruturação do frontend;
- TypeScript e JavaScript — desenvolvimento frontend;
- Node.js — ambiente de execução do frontend;
- Maven — gerenciamento de dependências backend;
- Git e GitHub — versionamento e gerenciamento do código-fonte;
- Visual Studio Code e IntelliJ IDEA — ambientes de desenvolvimento;
- Postman e Insomnia — testes de endpoints da API REST.

Referências Técnicas

- Documentação oficial do Spring Boot;
- Documentação oficial do Spring Security;
- Documentação oficial do PostgreSQL;
- Documentação oficial do Supabase;
- Documentação oficial do React;
- Documentação oficial do Next.js;
- Documentação oficial do Node.js;
- Documentação oficial do JWT (JSON Web Token);
- Princípios de modelagem de dados e construção de Diagramas Entidade-Relacionamento;
- Conceitos de autenticação, autorização e controle de permissões em aplicações web;
- Conceitos de APIs RESTful;
- Boas práticas de arquitetura de software e persistência de dados.
- Mercado Pago Developers — Documentação oficial. Disponível em: https://www.mercadopago.com.br/developers/docs. Acesso em: 08 out. 2026.

Repositório do Projeto

Repositório oficial do sistema Lion Fitness:

https://github.com/olucasortiz/estagio-supervisionado-monorepo